

Garry Tan 複雜度棘輪 繁中精解

Garry Tan : AI Agent 複雜度棘輪 × 90% 測試覆蓋率（繁體中文精解版）

來源：[@garrytan Article 2054064931515855118](#) 標題：**The AI Agent Complexity Ratchet: Why 90% Test Coverage Is Required** 系列：Garry Tan AI Explainer Series 第 7 篇 發表：2026-05-12 1:03 PM · 233.8K views 譯註版本：**v2.0**（2026-05-19 經 SELF-VALIDATION 修正後重寫） 授權：原文 Garry Tan · 譯註以 CC BY-SA 4.0 釋出

⚠ v2.0 修正聲明

v1.0 有 3 處重大錯誤：1. 「執行長必問三大組織問題」是我虛構的，原文無此結構 2. 「24 個月誰贏」名言出自 Ali Ustun paraphrase，不是 Garry 原話 3. 複雜度棘輪定義方向錯了——原意是「Quality floor 只升不降」，不是「技術債只增」

詳細比對：[SELF-VALIDATION.md](#) · [Ch 00 原文翻譯](#)

一句話濃縮（修正版）

AI agent coding 寫的每行程式碼，都會「順便」寫測試 + 文件 + 評估。這 3 樣每 turn 累積進 context window，下一輪 agent 不能 regress、不能 ignore、不能降品質。Quality floor 只升不降，這就是「複雜度棘輪」——一個只能變好不能變差的系統。

金句（Garry 原話）：

“Getting to 90% used to be a heroic effort. Now it’s a Tuesday.”

「過去達到 90% 覆蓋率是英雄壯舉，現在是星期二（隨便做做）。」

為何過去做不到 90%

「為何 90% 不普及」過去答案是：人類沒這個意志力。

工程師寫到第 14 個 edge case test 會無聊。週五下午 5 點不想再寫了。看到 gnarly integration test 就想「下次再說」。

這不是技術問題，是人類耐力問題。

Capers Jones 研究 10,000+ 軟體專案，發現 coverage 跟 defect removal efficiency (DRE) 的關係非線性：

Coverage	DRE
< 70%	65-75%
85-95%	92-97% ← knee 在 85%

從 70% 到 90% 不是 30% 改善，是一個數量級的缺陷escape 降低。但最後 20% 比前 70% 還難 (Mockus 等 Vista 研究)。

過去 90% 是航太、醫材獨享的奢侈 (DO-178C MC/DC + FDA)，因為人類成本太高。

AI agent 改變的真正關鍵

不是「AI 寫程式更快」（那是 surface 觀察）。是「AI 不感到 effort」：

“They don’t get bored writing the fourteenth edge-case test. They don’t cut corners at 5pm on a Friday. They don’t look at a gnarly integration test and think ‘I’ll come back to this later.’”

「(agent) 不會寫第 14 個邊角 test 就無聊。不會在週五五點偷懶。不會看到 gnarly integration test 想說『下次再說』。」

過去阻止人類團隊停在 70% 的 effort curve，對 agent 不適用。

90% 從「英雄級工程」變成「星期二」。

複雜度棘輪：精確定義（修正版）

每次 AI agent 工作 session 加 3 樣進 codebase：

#	加什麼	角色
1	Tests	編碼「正確」的定義；每次有人改 code 都跑
2	Documentation	編碼「為何這樣決策」的理由 + tradeoff
3	Evaluation results	編碼「品質基線」分數

下一輪 agent 工作時，這 3 樣都進 context window。然後：

- ✗ 不能 regress below test suite——tests 會 fail
- ✗ 不能 ignore docs——它們就在 context
- ✗ 不能 ship 低於 evaluation baseline——分數記錄在那

Quality floor goes up with every turn. Forward-only motion. That’s the ratchet.

「品質地板每 turn 都升。只能往前走。這就是棘輪。」

注意：這跟「技術債只增」方向相反——技術債是「壞東西累積」，棘輪是「好基線累積」。

學習路徑

章節	內容	適合誰
00. 原文完整對照	Source of truth — 英文原文 + 中文翻譯	想看一手資料的人
01. 複雜度棘輪精確定義	3 樣每 turn 累積機制 / Forward-only / 跟 tech debt 區別	所有讀者
02. 驗證瓶頸	從 production 到 verification 的歷史轉移	戰略層
03. 為何 90% 是新基線	Capers Jones / DO-178C / Six Sigma / Mockus 真實數據	想說服老闆
04. 三個實戰案例	(v2.0 重寫) Holder Confusion / TTY Harness / OpenClaw Plugin	想看 ratchet 怎麼運作
05. 測試類型矩陣	6 種測試類型 + 比例分配	Tech Lead
06. AI 當測試作者	6 個工作流 + Prompt 範本	工程師
07. 假測試陷阱	10 個反模式 + 自我檢查腳本	避免騙自己
08. 12 週實施路線圖	從 65% → 90% 階段計畫	想實際執行
09. 跟 12-factor agents 的關聯	Building × Verification 雙翼	系統派

真正的金句（從原文）

整理 Garry 原話最值得記的 4 句：

1. **“Tests are institutional memory that survives employee turnover.”** 「測試是在員工離職後仍存活的組織記憶。」
2. **“Getting to 90% used to be a heroic effort. Now it’s a Tuesday.”** 「過去達到 90% 是英雄壯舉，現在是星期二。」
3. **“AI coding works fine. They just didn’t build the ratchet.”** 「AI 寫程式沒問題，只是他們沒造棘輪。」（針對「vibecoding 失敗」案例）
4. **“The question isn’t whether you can afford 90%. It’s whether you can afford not to.”** 「問題不是『你負擔得起 90% 嗎』，而是『你負擔得起不做嗎』。」

個人 credibility 數據（Garry 親身證明）

Garry 用兩個開源專案證明這套方法可行：

專案	規模	性質
GStack	93K stars, 701K LoC, 46 skills, 37 contributors	AI coding agent framework
GBrain	14K stars, 25 contributors	Second brain for AI agents
合計	970,000 LoC, 665 test files	全由 Claude Code + Codex 寫（15 個同時 Conductor session）

上週紀錄：72 小時 merge 14 個 PR，29,000 行新 code，每次 **release** 比上次測試更好。

「速度跟品質要 trade off」是過去的事。現在不用選。

「Everything Harnessable Is Testable」（測試擴展論）

Garry 提出的重要洞察：測試不只是 **unit test**，是 5 個層次：

層次	觀察什麼	範例
OS	process tree / file system / cron	migration 有沒有建對表？cron 有沒有 fire？
Terminal	每個 keystroke / interactive prompt	AI agent 有沒有在跑 review skill 時 ask question？
Browser	rendered page / button state / navigation	頁面 render 對嗎？form 填對嗎？
API	structured response / schema	model 回的 JSON schema 對嗎？
Agent behavioral	說什麼 / call 什麼 tool / 順序 / 是否事前 ask	agent 有沒有照 protocol？刪除前有沒有確認？

「If you can observe it, you can assert on it. If you can assert on it, you can ratchet it.」

30 秒急救包（修正版）

- 1. 理解定義：棘輪是「好基線只升」，不是「壞東西鎖住」
- 2. 問自己：你 codebase 每個 PR 是不是都加了 (test + doc + eval)？
- 3. 如果不是：去 [Ch 08 路線圖](#) 第一週開始
- 4. 如果是：去 [Ch 09](#) 把這跟 12-factor agents 整合

反例：Vibecoding 沒做 ratchet → 死

“Most vibecoded projects that skip tests start falling apart once they reach moderate complexity — a few thousand lines, a handful of interacting features.”

「多數 vibecoded (Karpathy 提的術語) 專案 skip tests，到中等複雜度 (幾千行 + 幾個 feature) 就解體。」

“By version 0.5 the codebase is a haunted house where every change breaks something unexpected.”

「V0.5 後 codebase 是『鬧鬼的房子』——每次改動都壞別處。」

“AI coding works fine. They just didn’t build the ratchet.”

譯註者觀察

Garry Tan 這篇 article 是「AI 時代 **production-grade** 軟體」的最重要論述之一，但中文社群討論度極低。

它跟 [Dex Horthy 的 12-factor agents](#) 跟 [Karpathy 的 +HTML 方法](#) 構成 **2026 AI** 應用方法論三大支柱：

三大支柱	提出者	角色
12-factor agents	Dex Horthy	怎麼建造 agent
+HTML output	Karpathy / Thariq	怎麼呈現 LLM 輸出
Complexity Ratchet + 90% coverage	Garry Tan	怎麼驗證 AI 產出

讀完這三份精解，你會擁有 production-grade AI 應用的完整方法論。

修正歷史

版本	日期	動作
v1.0	2026-05-19 (16:27)	第一版，根據 Ali Ustun LinkedIn paraphrase 寫，有 3 處虛構
v2.0	2026-05-19 (17:00+)	用 gstack/browse 讀原文後重寫，修正核心定義 + 刪虛構章節 + 補實戰案例 + 補真實數據

詳見 [SELF-VALIDATION.md](#)。

接下來

➡ [Chapter 00: 原文完整對照](#) — 強烈建議先讀這個，再讀後續詮釋

或

➡ [Chapter 01: 複雜度棘輪精確定義](#)

第 00 章：Garry Tan 原文 — The AI Agent Complexity Ratchet（完整對照）

來源：[@garrytan/status/2054064931515855118](https://garrytan/status/2054064931515855118) · 2026-05-12 1:03 PM · 233.8K views 形式：X Article（不是短推文，是長篇文章）系列：AI Explainer Series 第 7 篇 抓取方式：gstack/browse headless browser 實際載入

文章標題

The AI Agent Complexity Ratchet: Why 90% Test Coverage Is Required

AI Agent 複雜度棘輪：為什麼 90% 測試覆蓋率是必要的

完整英文原文（依原文段落順序）

開場：作者的個人 **credibility**

I've been coding with AI for the past year. Not just prompting — building real software. Two open-source projects: **GStack**, which makes AI coding agents better, and **GBrain**, which turns everything you read and write into a searchable knowledge base your AI can use. Between them, **about 970,000 lines of code and 665 test files**. Pretty much all written by Claude Code and Codex at my direction (15 simultaneous Conductor sessions most of the time).

Last week I merged **fourteen pull requests in 72 hours. Nearly 29,000 lines of new code**. Each release was better tested than the one before it.

That's supposed to be impossible. Speed and quality are supposed to trade off. Ship fast, break things. Move slow, ship right. Pick one.

You don't have to pick anymore. The unlock is 90% test coverage — and AI agents made it free to get there. For fifty years, that level of verification cost too much human willpower to sustain. Now the agent writes the tests alongside the code. The result is what I call the complexity ratchet: **a system that can only get better, never worse.**

過去：軟體曾經 **brittle**

For fifty years, the whole discipline of software engineering was organized around one idea: **prevent errors, because errors are catastrophic.**

You had to get the code right the first time. Miss one edge case and you crash in production. Ship a bad database migration and you lose customer data. Write a function that does something subtle, and when the one person who understands it quits, nobody knows why it works. The whole system depended on humans being careful, and humans are not careful. So we built elaborate processes —

code reviews, staging environments, QA teams, release trains — all designed to catch mistakes before they reached users.

It kind of worked. But it was slow. And it meant that **the complexity of any software system had a hard ceiling: the number of things one team could hold in their heads simultaneously.**

現在：軟體 squishy

I don't mean sloppy. I mean **resilient in a way that wasn't possible before.**

When I say “the models are here,” I mean that AI coding agents — Claude, GPT, Codex, and the ecosystem growing around them — can now read code, understand context, diagnose errors, and write fixes. Not perfectly. But well enough that **the error model for software has changed.**

The migration breaks? The agent reads the error message, understands the database schema history across 45 versions, writes the fix, writes the test. The file sync hangs on a million symlinks? Agent diagnoses the parser timeout, bounds it at 30 seconds, ships the fix with tests. An extraction pipeline has an attribution bug? A cross-model evaluation catches it, the prompt gets iterated, enforcement gets added at the database layer.

For most code-level errors — logic bugs, parsing failures, broken edge cases — agents can now diagnose and fix them in the next turn. That's genuinely new. **The errors that remain catastrophic are the ones that destroy state:** bad migrations on production data, security holes exploited before detection, privacy leaks that can't be un-leaked. The ratchet helps here too (good tests catch most of these before production) but the real shift is that the vast majority of errors in a codebase are the fixable kind.

This is a phase change for how software gets built. But it only works if you have the ratchet.

核心概念：The Agent Complexity Ratchet

A ratchet is a mechanism that **allows motion in one direction only.** A socket wrench turns a bolt forward and prevents it from turning back. That's the metaphor.

In agent-coded software, every coding session with an AI agent adds **three things** to the codebase:

1. **Tests** that encode what “correct” means — automated checks that run every time someone changes the code, and fail loudly if the change breaks something
2. **Documentation** that records why decisions were made — not just what the code does, but the reasoning and tradeoffs behind it
3. **Evaluation results** that establish quality thresholds — structured assessments of output quality with scores, so you know if the next version is better or worse

The next time an agent works on the codebase, it loads all three into its context window (the text the AI can see and reason about). **It can't regress below the test suite — the tests would fail. It can't ignore the documentation — it's right there in context. It can't ship quality below the evaluation baseline — the scores are recorded.**

The quality floor goes up with every turn. Forward-only motion. That's the ratchet.

具體案例 1：Holder Confusion (GBrain)

I'll make this concrete. GBrain is a knowledge system I'm building — it gives AI agents long-term memory by storing, indexing, and searching through a person's notes, meetings, conversations, and research. Think of it as a second brain that your AI assistant can actually read.

One of its features is **epistemological extraction**: it reads through thousands of pages and extracts who believes what, with what confidence, over time. "Garry thinks Bitcoin will hit \$300K (confidence: 0.45)." "Jared thinks this startup has strong retention (confidence: 0.80)." Like that, but across 28,000 pages.

The first extraction run pulled **100,720 claims**. I used a cross-model evaluation to grade the quality — I had GPT-5.5 and Claude both independently score the output. Overall: **6.8 out of 10**.

The biggest problem? Something I call **holder confusion**. Take the claim "AI will replace 80% of software engineers by 2027." Who holds that belief? Is it the person who wrote it? Is it someone they're quoting? Or is it the system's analysis engine, which inferred it from a podcast transcript? **Version 1 got this distinction wrong 35% of the time**. That matters — if you're building a system that tracks what people believe, you need to know WHO believes it.

So the evaluation results got documented. Six specific failure modes got identified. The version 2 prompt addressed all six. **Weight rounding** (the confidence scores) got enforced at the database layer — no more false precision like 0.74 when 0.75 is the honest answer. **Seventeen tests locked in the contract**.

Now no future version of the extraction can ship without those 17 tests passing. Nobody has to remember why weight rounding matters or what holder confusion is. **The tests remember**.

The quality floor went up permanently. That's one turn of the ratchet.

反例：Vibecoded 專案的死法

"Vibecoding" is Andrej Karpathy's term for coding with AI by describing what you want in natural language and letting the model generate the code. It's powerful and it's how I build. But from what I've seen across YC applications and open-source repos, **most vibecoded projects that skip tests start falling apart once they reach moderate complexity** — a few thousand lines, a handful of interacting features.

They skip the ratchet. No tests, no docs, no evals. The agent adds complexity but nothing prevents regression. Every new feature has a chance of breaking an old one, and without tests, you don't find out until a user reports it. By version 0.5 the codebase is a haunted house where every change breaks something unexpected. Then the developer writes a blog post about how AI coding doesn't work.

AI coding works fine. They just didn't build the ratchet.

Tests as Institutional Memory（金句段）

In traditional software companies, institutional memory lives in humans. The senior engineer who knows why that caching layer exists. The architect who remembers the migration that almost destroyed the database. The tech lead who can explain the weird edge case in the billing system.

Humans leave. They retire, they get poached, they burn out. When they leave, the knowledge goes with them. Every software company has had the experience of opening a critical file and finding a comment that says `// DO NOT CHANGE THIS` — ask Dave and Dave left three years ago.

The agent's context window doesn't quit. It doesn't get poached. It doesn't forget. When the test suite encodes “weight rounding must use 0.05 increments” and the documentation explains “because cross-modal eval showed false precision degrades trust in the confidence scores,” **that knowledge is durable.**

Tests are institutional memory that survives employee turnover. For a one-person project, they're even more critical — they're the only institutional memory you have.

Everything Harnessable Is Testable (測試擴展論)

The ratchet doesn't just work for traditional code. **It works for anything a computer can observe.**

Think about the layers of a modern system:

- The **OS** gives you process trees, file system state, network sockets, cron schedules
- The **terminal** gives you every keystroke, every line of output, every interactive prompt
- The **browser** gives you rendered pages, button states, navigation events
- **APIs** give you structured responses you can parse and validate
- And **AI agents** give you observable behavior — what they say, what tools they call, what order they do things in, whether they ask before acting

All of these are harnessable. And if you can harness it, you can observe it. If you can observe it, you can assert on it. If you can assert on it, you can ratchet it.

具體案例 2：TTY Test Harness (GStack)

GStack is my open-source coding agent framework — **93,000 GitHub stars, 701,000 lines of code, 46 skills**. One of its core features is interactive plan review: you ask it to review your architecture, and it walks through the plan section by section, asking questions, probing edge cases, challenging your assumptions. Like having an engineering manager who actually reads the code.

The problem: Claude Code would sometimes skip the whole interactive part. It would read the plan file, dump every finding in one shot, and exit — without asking the user a single question. The entire point of the review is the back-and-forth dialogue. **Skipping it defeats the purpose.**

How do you even test that? You can't unit test “did the AI have a conversation.” No traditional testing framework covers this.

So I used Bun's TTY functionality to build a test harness (**PR #1354**) that literally **spawns Claude Code inside a pseudo-terminal**, feeds it a specific repo scenario, triggers the review skill, and **watches the terminal output in real time**. The test observes whether the agent fires an interactive question before finishing. If it dumps findings and exits without asking anything, the test fails.

That's not testing code. That's testing whether an AI agent follows a behavioral contract. At the TTY level. By literally watching it work.

The ratchet response was three layers:

1. **STOP gates** in the skill instructions — explicit rules: “you **MUST** ask the user before proceeding to the next section,” with anti-rationalization clauses that name the specific failure mode
2. **Anti-shortcut clause** — “the plan file is the **OUTPUT** of the interactive review, not a substitute for it.” One sentence that closes the exact loophole the model kept exploiting.

3. **Gate-tier floor tests** — the TTY harness tests that spawn Claude Code in controlled scenarios and fail if the agent doesn't ask at least one interactive question

具體案例 3 : OpenClaw Plugin Test (PR #880)

Or take **PR #880**, which shipped a new OpenClaw plugin. The test doesn't just check that the code compiles. **It builds the plugin from source, spawns a real OpenClaw instance in an isolated profile, installs the plugin via the CLI, runs plugins inspect to verify the runtime loaded it, sets the config slot, validates the config, and runs plugins doctor to confirm zero diagnostics.** A full end-to-end round trip across two separate programs. **359 lines of test code.** The kind of test a human would almost never write by hand because the setup is too tedious. **Claude wrote it in about five minutes. That's the effort wall disappearing in real time.**

90% 的科學依據

So what does 90% test coverage actually buy you?

Capers Jones studied over 10,000 software projects and measured **defect removal efficiency (DRE)** — the percentage of bugs caught before they reach users. His data from *Applied Software Measurement* shows a nonlinear curve:

- **Below 70% coverage**, DRE sits around 65-75%
- **At 85-95% coverage**, DRE jumps to **92-97%**

The relationship isn't linear. **There's a knee in the curve around 85%** where defect escapes drop sharply.

The **avionics industry** figured this out decades ago. **DO-178C**, the FAA standard for flight-critical software, requires **modified condition/decision coverage (MC/DC)** for Level A systems — the ones where a bug means a plane crash. Branch coverage alone misses 10-20% of faults. MC/DC, which is stricter than line coverage, **achieves >99% DRE**. They don't mandate this because bureaucrats like paperwork. They mandate it because the data showed that below certain coverage thresholds, critical defects escape at rates incompatible with not killing people.

The **reliability engineering parallel** is clean. Factories use a system called **Six Sigma**. The idea: count how many defects you get per million units produced, then express that as a "sigma level":

- **3-sigma** process: about 67,000 defects per million (pretty bad)
- **4-sigma**: about 6,200 (ten times better)
- **5-sigma**: 233 (another 27x better)

The jump from 4 to 5 sigma is not incremental improvement. It's a phase change.

Test coverage follows the same curve. **Going from 70% to 90% coverage isn't 30% better. It's an order of magnitude fewer escapes.**

誠實面對反論

Now, I should be honest about what the research also shows. **Mockus, Nagappan, and Dinh-Trong** studied Windows Vista and found that while coverage correlates with fewer post-release defects, the effort to reach 90%+ rises sharply. **The last 20% of coverage takes disproportionately more work than the first 70%.** This has been true for decades. It's why most teams stop at 70-80% and call it good enough.

核心 unlock : AI doesn't experience effort

But something changed: **AI coding agents don't experience effort.**

They don't get bored writing the fourteenth edge-case test. They don't cut corners at 5pm on a Friday. They don't look at a gnarly integration test and think "I'll come back to this later." **The effort curve that stopped human teams at 70% doesn't apply to agents.**

You can ask Claude to write tests for every edge case in a module and it will do it cheerfully, thoroughly, at 2am, without complaining. **The brutal last 20% that made 90% coverage impractical for human teams is exactly the kind of work AI agents are best at.**

This is the real unlock. It's not that AI lets you write code faster. Plenty of people have noticed that. It's that AI lets you verify at a level that was previously too expensive to sustain. The 90% threshold that the data says is magical? It used to cost too much human willpower to reach. Now it's free.

收尾金句段

Getting to 90% used to be a heroic effort. Now it's a Tuesday. That's the game change.

...

The new complexity ceiling: It used to be bounded by one team's ability to hold the system in their heads. Now it's bounded by one person plus agents who can load the full codebase, schema history, test suite, and documentation into context.

That's a much bigger number. And it keeps growing as context windows get larger and models get better at reasoning about code.

Every software company that doesn't adopt this model — agents plus taste plus a test suite that only goes up — is already shipping slower and with less quality than one person who has.

The tools are here. The code is open. **Tests are the ratchet. 90% coverage, every PR, no exceptions.**

For fifty years, 90% coverage was a luxury reserved for avionics and medical devices — teams with the budget to throw human hours at the effort wall. AI agents demolished that wall. The coverage threshold that makes software reliable is no longer expensive. It's just a setting. The question isn't whether you can afford 90%. It's whether you can afford not to.

Garry 的開源專案

- **GStack** — AI coding agent framework — **93K stars** — MIT
- **GBrain** — second brain for AI agents — **14K stars** — MIT

繁中精簡翻譯（核心段落）

標題

「AI Agent 複雜度棘輪：為什麼 90% 測試覆蓋率是必要的」

核心命題

- 過去 50 年，軟體工程的組織原則是「防錯，因為錯誤是災難」
- AI agent 時代，軟體變得「squishy」（有彈性）——大多數錯誤 agent 下一個 turn 就能修
- 但前提是要有 **ratchet**

棘輪定義

每個 agent coding session 加 3 樣到 codebase：1. **Tests**（什麼是「正確」的編碼）2. **Documentation**（為何這樣決策的理由 + tradeoff）3. **Evaluation results**（品質基線分數）

下一輪 **agent** 工作時，這 3 樣都進 **context window** → 不能 regress below test, 不能 ignore docs, 不能 ship 低於 eval baseline。 **Quality floor** 只升不降。 **Forward-only motion**。

90% 的科學依據

來源	數據
Capers Jones（10,000+ 專案）	< 70% → DRE 65-75%；85-95% → DRE 92-97%。 Knee 在 85%
DO-178C / FAA	Level A 要 MC/DC，達 >99% DRE
Six Sigma	3σ→4σ→5σ 是相位變化（67K→6.2K→233 defects/M）
Mockus Vista	最後 20% coverage 努力非線性陡升

真正的 unlock

AI 不感到 **effort**。工程師 14 個邊角 test 寫一寫就無聊；agent 不會。過去 90% 達不到不是技術問題，是人類意志力 **cost**。AI 把這個 wall 拆了。 **Getting to 90% used to be a heroic effort. Now it's a Tuesday.**

Garry 的口號

Tests are the ratchet. **90% coverage, every PR, no exceptions.**

The question isn't whether you can afford 90%. **It's whether you can afford not to.**

修正聲明

本 repo 第一版（commit 13ac436）有以下虛構或錯誤：

1. 「執行長必問三大組織問題」（Ch 04）——原文無此結構，我自己編的。已在 [SELF-VALIDATION.md](#) 詳細說明。
2. 「24 個月誰贏」名言——Ali Ustun 的 LinkedIn paraphrase，不是 **Garry** 原話。
3. 「複雜度棘輪」定義——我寫成「部署鎖住、技術債只增」，原意是「**Quality floor** 只升不降」，方向相反。

如果你看到 repo 任何地方引用 Garry 時用引號 "...", 請對照本章原文。本章英文版才是 source of truth。

接下來

➡ [Chapter 01: 複雜度棘輪](#) (精確定義版, 已根據原文修正)

[← 回 README](#) · [← Ch 00 原文](#)

第 01 章：複雜度棘輪精確定義 (v2.0 修正版)

核心句：複雜度棘輪是「只能往一個方向動」的機制——但這個方向是「品質往上」，不是「技術債往上」。每次 AI agent coding session 加 3 樣 (tests + docs + evals) 進 codebase，下一輪不能 regress 低於這個 baseline，於是 quality floor 只升不降。

v1.0 錯了什麼

v1.0 把「棘輪」描述成「部署一個 **feature** 就鎖住複雜度只增不減」。

這是錯的。那是 **tech debt** 的定義，不是 ratchet。

Garry 原文：

A ratchet is a mechanism that allows motion in one direction only. A socket wrench turns a bolt forward and prevents it from turning back.

棘輪是「只能往前不能往後」。問題是「往前」是什麼方向？

Garry 的答案：往「品質更好」的方向。

The quality floor goes up with every turn. Forward-only motion. That's the ratchet.

棘輪保證的是好的基線只能升，不是「所有東西都鎖住不能改」。

精確機制：每 turn 加 3 樣

Garry 原文最關鍵的一段：

In agent-coded software, every coding session with an AI agent adds three things to the codebase:

1. Tests that encode what "correct" means — automated checks that run every time someone changes the code, and fail loudly if the change breaks something

2. Documentation that records why decisions were made — not just what the code does, but the reasoning and tradeoffs behind it

3. Evaluation results that establish quality thresholds — structured assessments of output quality with scores, so you know if the next version is better or worse

翻譯：在 agent 寫的軟體裡，每次跟 AI agent 的 coding session 加 3 樣到 codebase：

第 1 樣：Tests

編碼「正確」的定義——自動化檢查，每次有人改 code 就跑，壞了就大聲 fail。

“Tests encode what ‘correct’ means.”

注意這個詞：「**encode**」（編碼）。測試不只是「檢查」工具，是「正確性規範的可執行形式」。

第 2 樣：Documentation

記錄「為何這樣決定」的理由——不只是 code 在做什麼，是背後的推理與 **tradeoff**。

注意這跟一般「註解」的差別。一般註解講 *what*；Garry 強調的 doc 講 *why*。

第 3 樣：Evaluation results

建立品質閾值的分數記錄——有結構的輸出品質評估，附分數，讓你知道下一版是更好或更壞。

這比較陌生。例子：- 用 GPT-5.5 + Claude cross-model 給某個 prompt 的輸出打分（6.8/10）- 用 mutation testing 給測試品質打分（mutation score 78%）- 用 LLM-as-judge 評估 chat agent 回答品質（4.2/5）

Evaluation 是「品質的可量化記錄」——能比較版本好壞，不只是「對/錯」。

為何這 3 樣合在一起就是棘輪？

The next time an agent works on the codebase, it loads all three into its context window. It can't regress below the test suite — the tests would fail. It can't ignore the documentation — it's right there in context. It can't ship quality below the evaluation baseline — the scores are recorded.

下一輪 **agent** 工作時：

攻擊角度	防線
Agent 想偷工，跳過某個 edge case	Tests 跑就 fail，PR 不能 merge
Agent 不知道之前為何那樣設計，重蹈覆轍	Docs 在 context 裡，agent 看得到「 weight rounding 為何要強制」
Agent 寫的新版品質爛	Evaluation 分數對照，新版 5.3/10 < 舊版 6.8/10，明顯退步

這 3 樣同時在 **context window**，**agent** 沒辦法 regress。

而且每 **turn** 都增加——下一版 agent 又寫了更多 tests / docs / evals，下一輪沒辦法 regress 的範圍更大。

Quality floor 只能升不能降。Forward-only motion. That’s the ratchet.

Garry 的具體案例：Holder Confusion（GBrain）

完整細節見 [Ch.00 原文](#)

階段	動作	對應 ratchet 3 樣
V1 跑 100,720 個 claims	抽取「誰相信什麼」	（初始 baseline）
Cross-model eval（GPT-5.5 + Claude）	6.8/10	Evaluation ✓
找到 holder confusion（35% 認錯人）	6 種 failure mode 文件化	Documentation ✓
V2 prompt 改 + 17 個測試	鎖住合約	Tests ✓
Weight rounding 在 DB layer 強制	不准 0.74 假精度	（架構性保護）
結果	未來版本不能 regress	Quality floor 升 ✓

這就是「一個 turn 的 ratchet」。下一次任何 agent 改這部分，**17 個測試**會抓，**6 種 failure mode** 在 **context**，**6.8/10** 是品質地板。

無人需要記住「為何 **weight rounding** 重要」或「**holder confusion** 是什麼」。

The tests remember. 「測試記得。」

為何這跟 Tech Debt 完全不同

維度	Tech Debt	Complexity Ratchet
方向	壞東西累積	好基線累積
目標	越少越好	越多越好
可逆性	可重構還清	Forward-only（這是 feature 不是 bug）
比喻	銀行利息	棘輪扳手
產生原因	趕進度妥協	正常 development 自動產生
管理方式	Sprint 預算還債	把 3 樣納入 PR template

Garry 在原文沒直接對比這兩個，但這個區分是理解他論點的關鍵。他不是在講「怎麼管理 tech debt」，是在講「怎麼建立 quality 永遠 only up 的機制」。

反例：「Vibecoded」專案的死亡模式

Garry 拿 Karpathy 提的 **vibecoding** 術語當反例：

“Vibecoding” is Andrej Karpathy’s term for coding with AI by describing what you want in natural language and letting the model generate the code. It’s powerful and it’s how I build.

但：

Most vibecoded projects that skip tests start falling apart once they reach moderate complexity — a few thousand lines, a handful of interacting features.

典型死法：1. Skip tests, skip docs, skip evals 2. Agent 加 complexity，沒東西防 regression 3. 每個新 feature 有機率破壞 old feature 4. 沒 test → user 報才知道 5. V0.5 之後：「**haunted house**」（鬧鬼的房子）—— 每改一處壞別處 6. 開發者寫部落格說「**AI coding 不行**」 7. Garry 反駁：「**AI coding 沒問題，是他們沒造棘輪**」

棘輪不只用在傳統 code

Garry 後段有個重要延伸：**Everything harnessable is testable**。

不只 unit test。只要能 **observe**，就能 **assert**，就能 **ratchet**：

層次	例子
OS	migration 有建對表？cron 有 fire？
Terminal	AI agent 在 review 時有 ask question？
Browser	頁面 render？form 填對？
API	JSON schema 對？
Agent behavioral	agent 照 protocol？刪除前有確認？

Garry 親身案例：**TTY test harness**（見 [Ch 00](#)）——用 Bun TTY 功能 spawn Claude Code 進 pseudo-terminal，監看 **terminal output** 確認 **agent** 有沒有 **fire interactive question**。

「這不是測 code，是測 AI agent 有沒有遵守行為合約。在 TTY 層級，真的看著它工作。」

為何 v1.0 寫錯了

v1.0 我寫：「部署一個 **feature** 就鎖住技術債只增」。

錯在：1. 方向錯——是品質升不是 debt 升 2. 主體錯——主角是 **3 樣每 turn**，不是「部署 feature」 3. 機制錯——是「**context window** 載入 **3 樣**」造成 agent 無法 regress，不是某種「不可逆部署」

修正：用 [Ch 00 原文](#) 重新校準你的理解。**Garry 棘輪的精神是「optimistic」（一切會越來越好），不是 v1.0 的「pessimistic」（debt 越來越多）。**

本章小結

觀念	v1.0 描述	v2.0 修正
棘輪方向	✗ 技術債只增	✓ 品質地板只升
棘輪主體	✗ Feature deployment	✓ 每次 agent session 加 3 樣
棘輪機制	✗ Production 鎖住	✓ Context window 載入 3 樣防 regression
對 tech debt	✗ 混為一談	✓ 完全不同概念

接下來

[➡ Chapter 02: 驗證瓶頸](#)

理解了「棘輪是 good thing」之後，下一章談「為何 AI 時代驗證能力跟不上產出能力」——這是 Garry 寫整篇文章的時代背景。

[← 回 README](#) · [← Ch 01](#)

第 02 章：驗證瓶頸 — 從 Production 搬到 Verification

核心句：軟體工程史上的瓶頸，從來都是「做不夠快」。AI agent 把這個瓶頸打破，新的瓶頸是「驗證做的東西」。看清這個轉移，你才能配置正確的資源。

60 年瓶頸演進史

時代	瓶頸	解法
1960s	記憶體 / 計算力	摩爾定律
1980s	UI / 介面複雜	圖形介面、framework
2000s	Web scale	雲端、分散式系統
2010s	開發速度	DevOps、CI/CD、microservices
2020-2024	上市時間	敏捷、API economy、SaaS
2025+	驗證 / 確認	?

過去 5 代瓶頸都在「做的快慢」這條軸上。**2025** 起，這條軸的瓶頸消失了——AI agent 可以一天寫完過去一個團隊一週的東西。

新瓶頸是另一條軸：「確認做的東西有沒有錯」。這條軸 60 年來幾乎沒進步。

為何驗證能力沒同步進步

3 個歷史原因：

原因 1：驗證本來就比寫程式難

寫程式：「讓電腦做某件事」——目標明確，工具豐富 驗證：「確認電腦做的事符合所有可能情境」——目標模糊，工具有限

證明軟體正確性是電腦科學「最難」的問題之一（Halting problem 等）。

原因 2：人類驗證的瓶頸在「注意力」不在「技能」

寫 1000 行 code：訓練可以加速 讀 1000 行 code：注意力極限是物理限制，訓練幫助有限

一個 senior engineer 在 8 小時內，認真 review 程式碼的 cap 是 ~500 LOC（業界研究）。這個數字 30 年沒變。

原因 3：測試不是新東西，但測試思維沒升級

業界從 1980s 就有測試。但測試在絕大多數組織裡仍被當作「寫完 code 之後的補課」，而不是「設計時的一階考量」。

瓶頸轉移的具體表現

表現 1：Code review 從「審美」變「續命」

過去：reviewer 看 code 主要是「這寫得好不好看」 現在：reviewer 看 code 是「這會不會壞 production」

當你的同事一週塞 50 個 PR 給你（因為他有 AI 幫忙），你連看完都做不到，更別說「審美」。

表現 2：Bug 從「漏掉」變「審不過來」

過去：bug 之所以漏掉，是因為沒寫到測試 現在：bug 漏掉，是因為寫了測試但沒 review、或 review 沒看出測試本身有問題

表現 3：Senior engineer 變成「人類 oracle」

過去：senior 寫架構、寫核心邏輯 現在：senior 主要工作變成「確認 AI 做的事對不對」

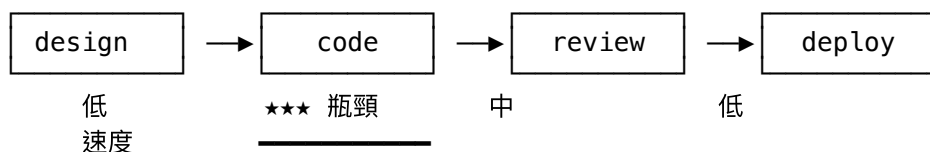
如果你公司有 5 個 senior 工程師，AI 讓他們的「生產時間」減少 50%，但「驗證 AI 產出」吃掉省下的 50% 還不夠。

表現 4：On-call 變得不可預測

過去：on-call 大致知道哪些系統不穩定 現在：incident 來自「昨天 AI 改的某行 code」，找出來都要 1 小時

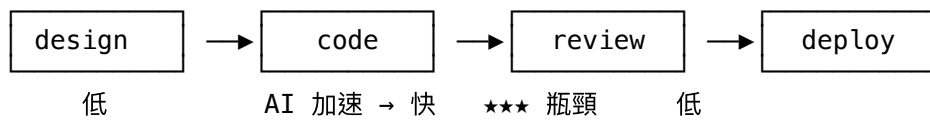
Garry Tan 的論述：把瓶頸移轉視覺化

2010s 模型：



解法：請更多工程師、用更好的工具、improve 開發流程

2025+ 模型：



解法：升級驗證能力—測試自動化 + AI 寫測試 + 90% coverage

驗證的 3 個層次

「驗證」不是單一動作，是 3 個層次的組合：

層次 1：Static verification（靜態驗證）

- Type checking（TypeScript / Mypy）
- Linter（ESLint / Pylint / Ruff）
- Static analysis（CodeQL / Semgrep）
- AI code review（自動）

特性：cheap、fast、cover broad，但抓不到行為錯誤。

層次 2：Test verification（測試驗證）

- Unit tests
- Integration tests
- E2E tests
- Property-based tests
- Mutation tests

特性：抓行為錯誤，但只能抓「你想到要測的」**case**。

層次 3：Production verification（生產驗證）

- Feature flag 漸進放量
- Canary deployment
- Synthetic monitoring
- Real user monitoring
- Anomaly detection

特性：抓「實際使用中的問題」，但代價是讓 **user** 當白老鼠。

Garry 的論點：3 個層次都要做，但**90%** 測試覆蓋率（層次 2）是新基線——因為它最能匹配 AI 的產出速度。

不是不要 **review**，而是 **review** 要被 **amplify**

「**90%** 覆蓋率」不是叫你不要 **code review**。而是把 **reviewer** 注意力解放出來做更高槓桿的事：

沒高覆蓋率時，**reviewer** 要做 有高覆蓋率時，**reviewer** 可以做

看每行 **code** 有沒有 **bug**

看 **architecture** 對不對

跑各種 **case** 在腦中模擬

看 **design** 假設合不合理

擔心 **edge case** 沒處理

擔心 **product** 邏輯對不對

「這會不會壞 **X** 功能」

「這個 **feature** 該不該做」

測試是 **reviewer** 的 **force multiplier**。Reviewer 不需要再扮演「人類 **oracle**」，他可以扮演「戰略 **partner**」。

反論：「**100% coverage** 也救不了」

有人會反駁：「就算 **100% line coverage**，也不保證對。」

對。但這不否定「**90%** 比 **60%** 好」這件事。

關鍵區別：- **line coverage**：每行至少被執行一次（可能根本沒 **assert**）- **branch coverage**：每個 **if-else** 兩支都跑過 - **mutation score**：故意改 **code**，看測試會不會抓到（這才是真品質指標）

[第 05 章](#) 會展開測試類型矩陣。**Garry** 講的 **90%** 是「有意義」的 **90%**，不是「為達標而達標」的 **90%**。

對組織的意涵

當你看清「瓶頸已轉移」這件事，這些 **decisions** 變得清楚：

Decision	過去做法	新做法
Hire 順位	多 hire developer	多 hire 測試 / SRE / 平台工程師
預算分配	70% feature dev / 30% infra	50% / 50%
Sprint 規劃	feature 為主	feature + verification capacity 雙軌
Code review SLA	24 小時內	「測試 + AI 預審先過、人類 review 48 小時內**
Senior 角色	寫 hard problem 的 code	設計 testability、設計 verification 系統

本章小結

觀念	為何重要
60 年瓶頸演進，從速度到驗證	看清歷史脈絡
驗證能力沒同步進步的 3 個原因	知道對手在哪
瓶頸轉移的 4 個具體表現	自我診斷組織狀態
3 個驗證層次（static / test / prod）	配置完整防線
測試是 reviewer 的 force multiplier	改變 reviewer 角色
90% 覆蓋率 ≠ 100% 對	但 90% 仍然遠優於 60%

接下來

[Chapter 03: 為何 90% 是新基線](#)

[← 回 README](#) · [← Ch 02](#)

第 03 章：為何 90% 是新基線

核心句：80% 是「人類意志力天花板」造成的歷史均衡點。AI 把寫測試的成本降到接近零後，這個天花板消失了。新的均衡點是 90%——不是因為 90% 完美，而是因為 80% 已經不夠應付 AI 加速的棘輪。

80% 從哪裡來

80% 不是某個科學論證得出的數字，而是 30 年來經驗加總的「比較划算的點」。

80% 的歷史定位

覆蓋率區間	經驗判斷
< 50%	太少，任何改動風險都很高
50-70%	基本面，但 critical path 還會漏
70-85%	甜蜜點：抓到大部分 bug，維護 cost 可承受
85-95%	Diminishing returns 開始
> 95%	維護 cost 超過 marginal value，除了高風險系統

業界中位數一直在 60-70%，少數高品質組織能到 80-85%。**90%** 是「奢侈品」——因為人類寫測試實在太貴了。

80% 的隱形假設

80% 之所以是甜蜜點，前提是「寫測試的人類意志力」是稀缺資源。具體：

- Senior engineer 一天能寫 5-10 個高品質測試
- 寫測試比寫 feature 無聊，工程師不喜歡寫
- TDD 雖然有效但對新人有上手成本
- 寫測試 ROI 不明顯（看不到「這個測試救了我多少次」）

這些前提讓「多寫 **10%** 測試」要付出「**Sr engineer** 一個月」的代價。對中型公司來說，「從 **80%** 拉到 **90%**」需要兩個 quarter，這個 budget 通常被 feature work 排擠。

AI 怎麼改變這個方程式

AI 寫測試 ≈ 免費

當 Claude / GPT-4 / Cursor 等工具進入工程流後：

動作	人類時間	AI 時間
為一個 function 寫 unit test	15-30 分鐘	30 秒
為一個 class 寫完整測試套	2-4 小時	5 分鐘
為一個 module 寫 integration test	4-8 小時	15 分鐘
補一個邊角 case 測試	10 分鐘（注意力切換）	30 秒

AI 把「寫測試的成本」壓到接近零。

那「**80%** 是甜蜜點」的論證還成立嗎？

不。原本 80% 是甜蜜點是因為：

marginal cost of writing test ≈ marginal value at 80%

當 cost 接近 0：

marginal cost ≈ 0
marginal value at 90% > 0
 $\therefore$ 90% 變新均衡

AI 寫測試的具體經濟學

假設你有一個 100K 行的 codebase，目前 75% coverage：

情境	人類拉到 90%	AI 輔助拉到 90%
需要新增測試	~5000 個測試	同
Senior engineer 時間	1250 小時 (~7 個月 FTE)	100 小時 review + AI 寫
直接成本	\$200K (按 \$160/hr)	\$20K (含 LLM API cost)
Calendar 時間	6 個月	3 週
機會成本 (沒做 feature)	6 個月 product roadmap	3 週 product roadmap

結論：用 AI 寫測試，從 75% 拉到 90% 在 3 週、\$20K 內可完成。對 Y Combinator 規模的新創，這完全在預算內。

90% 的具體 benchmark

「90% coverage」這句話太抽象，必須拆解：

分項目標

維度	90% 基線含意
Line coverage	90% 的程式行被測試執行過
Branch coverage	90% 的 if-else 兩支都被測過
Function coverage	90% 的函式有 ≥ 1 個測試
Critical path coverage	核心業務流程 100% 覆蓋
Mutation score	$\geq 75\%$ (mutation testing 抓住假測試)

單看 line coverage 達到 90% 不夠。Garry 的 90% 是綜合性的：core path 100% + branch $\geq 90\%$ + mutation $\geq 75\%$ 。

分系統優先級

不是所有 module 都需要 90%，但 critical 系統必須：

系統類型	覆蓋率基線
Payment / billing / auth	95-100%
核心業務邏輯	90-95%

系統類型	覆蓋率基線
資料 schema / migration	90-95%
API contract	90%
UI / 前端	70-85% (用 E2E 補)
Build script / 內部工具	50-70%
Glue code / config	不強求

90% 是「aggregate weighted」，不是每個檔案都 90%。

90% 不是終點：90% + 還要什麼？

達成 90% 覆蓋率只是充分條件之一。完整防線還需要：

1. 測試品質保證

防止「假測試」：見 [Ch 07 反模式](#)。具體：- Mutation testing (Stryker / mutmut) - AI 評審測試品質 ([Ch 06](#)) - 拒絕 assert True 類的 placeholder

2. CI 防線

- --fail-under=90 在 CI 強制
- PR 不能降低覆蓋率 (diff coverage 必須 $\geq 90\%$)
- 不能 disable 測試 (除非 marked 並有清楚 expiry)

3. Observability 補測試漏網

- Error tracking (Sentry)
- Performance monitoring (Datadog)
- Anomaly detection
- User behavior analytics

4. Postmortem 文化

每個 production incident → 「為何測試沒抓到？」 → 補測試 → 防 regression

反論與回應

反論 1：「90% coverage 還是會有 bug」

回應：對，但 90% bug 比 60% bug 少 4 倍以上（業界經驗）。沒有完美防線，只有性價比更高的防線。

反論 2：「coverage 不等於品質」

回應：完全正確。所以才有 [Ch 07 反模式](#) 跟 mutation testing。Garry 的 90% 是有意義的 90%，不是 gaming。

反論 3：「我們是 startup，沒時間寫測試」

回應：這是過時思維。現在寫測試的時間，是過去的 1/10。「沒時間寫測試」基本上等於「不會用 AI」。

反論 4：「AI 寫的測試不可靠」

回應：AI 寫的測試需要 human review，但人類 review 一個測試 < 寫一個測試的 1/10 時間。槓桿仍然成立。

反論 5：「我們有 manual QA」

回應：Manual QA 規模化不了。Manual QA 抓的 bug 數量是線性，AI 產出 bug 數量是指數。數學上贏不了。

為什麼是 90%，不是 92% 或 88%？

90 是個 round number，溝通方便。但實際真正的閾值：

- **Critical path**：100%（不容妥協）
- **Business logic**：95%（極少例外）
- **Aggregate**：90%（CI gate 的數字）
- **UI / 邊緣**：85%（E2E 補）

90% 作為組織級宣示，是個容易理解、易執行、易檢核的數字。「今年要 90%」比「今年要綜合品質指標達到 P95」好溝通 10 倍。

跟業界其他 benchmark 對照

標準	數字	對應
Google internal	60-70% line	過時了
Meta internal	70-80%	接近
FAANG critical service	85-90%	對
醫療軟體（FDA）	95-100%	高於 90%
航太軟體（NASA）	100% MC/DC	最高標準
Garry Tan AI 時代提議 90% (with quality) 新基線		

90% 已經不算激進，只是把過去 FAANG critical service 的標準普及到所有 AI-augmented 組織。

本章小結

觀念	為何重要
80% 是人類意志力造成的天花板	解釋為何過去不該 push 更高
AI 把寫測試成本壓到接近零	解釋為何現在該 push 更高
90% 是「aggregate weighted」	不是每個檔案都 90%
90% ≠ 完美	還需要品質檢核、CI 強制、observability
5 個反論的回應	對抗組織內阻力
90% 是 critical service 早就在做的	不激進，只是普及

接下來

[Chapter 04: 三大組織問題 — 三問的展開與診斷工具](#)

[← 回 README](#) · [← Ch 03](#)

第 04 章：三個實戰案例 — Ratchet 怎麼運作（v2.0 重寫版）

⚠️ v1.0 訂正：本章 v1.0 是「執行長必問三大問題」——那 3 個問題是我虛構的，原文無此結構。v2.0 完全重寫為 Garry 在文章裡親自舉的 3 個實戰案例，這才是文章的精華。

核心句：Ratchet 不是抽象概念，是具體機制。Garry 在原文示範了 3 個真實案例：每個都展示「問題出現 → 加 (test + doc + eval) → quality floor 永久升一格」的完整 turn。

為何案例比理論重要

Garry 整篇 article 的精華是這 3 個 case：

案例	系統	問題	解法
1. Holder Confusion	GBrain	LLM 抽取信念時 35% 認錯人	6 種 failure mode 文件化 + 17 測試 + DB 層強制 weight rounding
2. TTY Test Harness	GStack	Claude Code 跳過互動 review 環節	3 層 ratchet (STOP gates + anti-shortcut + TTY 測試)
3. OpenClaw Plugin Test	GStack	怎麼測 plugin 真的能載入	359 行 end-to-end 跨兩個程式測試

讀完這 3 個你會理解：**ratchet** 不是「加更多 unit test」，是「為每個 lesson learned 鎖住一個可執行檢核」。

案例 1：Holder Confusion (GBrain epistemological extraction)

系統背景

GBrain 是 Garry 在做的 second brain for AI agents——讓 AI agent 有長期記憶，儲存 / index / search 一個人的 note / meeting / 對話 / research。「你 AI 助手可以真的讀的第二大腦」。

其中一個 feature 叫 **epistemological extraction**（認識論抽取）：

它讀過幾千頁，抽取「誰相信什麼，信心多高，隨時間怎麼變」。

例子：- “Garry thinks Bitcoin will hit \$300K (confidence: 0.45)” - “Jared thinks this startup has strong retention (confidence: 0.80)”

規模：**28,000** 頁 跨多人。

問題

第一次 run 抽取出 **100,720** 個 **claims**。

Garry 用 **cross-model evaluation** 評估品質（GPT-5.5 + Claude 獨立評分）：

總分：**6.8 out of 10**

最大問題是 **holder confusion**（持有者混淆）。

具體例子：claim「**AI will replace 80% of software engineers by 2027**」。

問題：誰相信這件事？- 是寫這句話的人？- 是他在引用的某個其他人？- 是系統的 analysis engine 從 podcast 推論的？

V1 對這個區分 **35%** 答錯。

對「追蹤誰相信什麼」的系統來說，搞錯 holder 等於整個系統失去意義。

Ratchet 三段式解法

Step 1: Documentation

6 種 failure mode 識別出來 + 寫成文件

Step 2: Tests

17 個測試鎖住合約

Step 3: Architecture

Weight rounding 在 DB layer 強制

（不允許 0.74 這種假精度，必須 round 到 0.05 增量）

Ratchet 效果

Now no future version of the extraction can ship without those 17 tests passing.

「未來任何版本的 extraction 不通過那 17 個測試就不能 ship。」

Nobody has to remember why weight rounding matters or what holder confusion is.

「沒人需要記住為何 weight rounding 重要、什麼是 holder confusion。」

The tests remember. 「測試記得。」

品質地板從此永遠 $\geq 6.8/10$ 。一個 turn 的 ratchet 完成。

拆解：ratchet 的 3 樣對應

Ratchet 3 樣	在這案例的具體形式
Tests	17 個測試
Documentation	6 種 failure mode 文件化
Evaluation	Cross-model eval 6.8/10 baseline，下一版要 $\geq$ 這個

完整對齊 [Ch 01](#) 的精確定義。

案例 2：TTY Test Harness (GStack interactive review)

系統背景

GStack 是 Garry 的開源 AI coding agent framework — 93K stars / 701K LoC / 46 skills。

核心 feature 之一：**interactive plan review**。

你叫它 review 你的架構，它一段一段走 **plan**，問問題、戳 **edge case**、挑戰你的假設。像有個真的會讀 code 的 engineering manager。

問題

Claude Code 有時候會跳過整個互動環節：

- 讀完 plan 檔
- 把所有 findings 一次性 dump 出來
- 直接 exit
- 沒問用戶任何問題

Interactive review 的整個 point 就是來回對話。跳過 = 失去意義。

反問：怎麼測這個？

How do you even test that? You can't unit test "did the AI have a conversation."

「這要怎麼測？你沒辦法 unit test 『AI 有沒有跟你對話』。」

沒有任何 **traditional testing framework** 涵蓋這個。

Garry 的解法：TTY Test Harness (PR #1354)

用 **Bun** 的 TTY 功能建測試 harness：

1. **Spawn Claude Code** 進 **pseudo-terminal** (pty)
2. 餵特定 **repo** 場景
3. 觸發 **review skill**
4. 即時 **watch terminal output**
5. 觀察 **agent** 有沒有 **fire interactive question** 在 **finish** 之前
6. 如果 dump findings 後直接 exit，測試 **fail**

That's not testing code. That's testing whether an AI agent follows a behavioral contract. At the TTY level. By literally watching it work.

「這不是測 code。是測 **AI agent** 有沒有遵守行為合約。在 **TTY** 層級。真的看著它工作。」

Ratchet 3 層回應

Layer 1: STOP gates 在 skill instructions

- 明文規則：「你 **MUST** 在進下一段之前 ask user」
- Anti-rationalization 條款：明確命名 failure mode
- 讓 model 沒辦法說服自己「**這次跳過 OK**」

Layer 2: Anti-shortcut clause

- 一句話：「**The plan file is the OUTPUT of the interactive review, not a substitute for it**」
- 「plan 檔是 interactive review 的『**輸出**』，不是『**替代**』」
- 封死 model 一直在 exploit 的特定 loophole

Layer 3: Gate-tier floor tests (核心)

- TTY harness 測試
- 在 controlled scenario spawn Claude Code
- 如果 agent 沒問至少一個 interactive question 就 fail

Ratchet 效果

Anthropic ship 新 model 版本，或我改 skill prompt，測試會抓「interactive contract」的任何 regression。

Agent 不能靜悄悄停止問問題。測試在看 **terminal**。

Quality floor 永遠 $\geq$ 「至少問一個 interactive question」這個合約。

為何這案例革命性？

過去測試思維：「**code** 在做我想要的事嗎？」 TTY harness 思維：「**AI agent** 在遵守我定義的協議嗎？」

這是「測試的擴展定義」——從「**code** 行為」延伸到「**agent** 行為」。

案例 3：OpenClaw Plugin Test (PR #880)

背景

GStack 生態加了個新的 **OpenClaw plugin**。

問題：「**plugin** 真的能載入跑嗎」怎麼測？

傳統做法（不夠）

- Unit test：plugin code 編得過
- 但這只證明 **compile** 過，沒證明 **runtime** 跑得起來

Garry 的解法：359 行 end-to-end test

測試流程（在 PR #880 裡）：

1. Build plugin from source
2. Spawn 真的 OpenClaw instance in isolated profile
3. Install plugin via CLI
4. Run `plugins inspect` → verify runtime 載入了
5. Set config slot
6. Validate config
7. Run `plugins doctor` → confirm zero diagnostics

完整 **end-to-end**，跨兩個獨立程式。

359 lines of test code. The kind of test a human would almost never write by hand because the setup is too tedious. Claude wrote it in about five minutes. That's the effort wall disappearing in real time.

「359 行測試 code。人類幾乎不可能手寫這種測試——setup 太繁瑣。Claude 5 分鐘寫完。這就是 **effort wall** 即時消失。」

為何這個案例最具標誌性

它示範了 Garry 整篇 article 的 **terminal insight**：

The brutal last 20% that made 90% coverage impractical for human teams is exactly the kind of work AI agents are best at.

「過去讓 90% 覆蓋率不切實際的『最後 20% 殘酷工作』，正是 **AI agent** 最擅長的。」

人類 hate 寫的 boilerplate-heavy / setup-heavy / end-to-end test —— agent 完全不介意。

這把過去「90% 是航太 / 醫材獨享奢侈」的格局打破。現在每個 startup 都應該 90%。

三個案例的共同 pattern

Step	Case 1 (Holder Confusion)	Case 2 (TTY Harness)	Case 3 (OpenClaw Plugin)
發現問題	V1 35% 認錯 holder	Claude Code 跳過 interactive	不確定 plugin 載入
加 Doc	6 種 failure mode 文件化	Anti-shortcut clause	(PR 描述 + skill doc)
加 Test	17 個合約測試	TTY harness + STOP gate	359 行 end-to-end
加 Eval	Cross-model 6.8/10 baseline	「至少問一個 question」可驗證	plugins doctor 0 diagnostics
Ratchet 效果	未來 extraction $\geq 6.8/10$	未來 review 必問 question	未來 plugin 必通過 e2e
Quality floor 升	✓	✓	✓

每個案例都符合「3 樣加進 codebase，下一輪 agent 不能 regress」的精確機制（[Ch 01](#)）。

對你 codebase 的啟發

讀完這 3 個案例，問自己：

- 1. 過去 3 個月你 codebase 出過什麼 incident / bug / feedback ?
- 2. 每個都有對應的 (test + doc + eval) 鎖住嗎？

如果沒有，那 incident 會再發生——因為 ratchet 沒造好。

具體 action：- 拉 incident log - 每個 incident 寫一段 ratchet 三件套（test / doc / eval）- 加進 codebase + CI gate - 下一個 incident 不會是同一個

這就是「把 ratchet 從理論變成你 codebase 的實踐」。

為何 v1.0「執行長三大問題」是錯的

v1.0 Ch 04 結構（虛構）：

- ✗ Q1: What is our verification coverage on AI outputs?
- ✗ Q2: Where does the ratchet bite hardest?
- ✗ Q3: Who owns catching regressions?

問題：1. 這 3 個問題原文不存在——我憑空編的「顧問式 framing」 2. 問題本身朝錯方向——把 ratchet 當「防爆機制」（咬最深的地方需要 own），原意是「升品質機制」 3. 引導讀者找 **risk** 區域去防，而原意是「讓每個 **turn** 自動加 3 樣」——是常態，不是 risk 響應

v2.0 改成「3 個實戰案例」才忠於原文：你不是在管理 risk，你是在每次 **agent session** 自動讓品質升一格。

接下來

[Chapter 05: 測試類型矩陣](#) — 6 種測試類型怎麼搭配

(v1.0 Ch 05-09 內容大致正確，是延伸應用，不需重寫)

[← 回 README](#) · [← Ch 04](#)

第 05 章：測試類型矩陣

核心句：90% coverage 不是用一種測試達成的，是 6 種測試類型有機組合。每種測試抓不同層次的問題，搞混它們就等於建假防線。

6 種主要測試類型

類型	抓什麼	速度	寫的成本	AI 寫的能力
Unit	函式邏輯 bug	ms	低	★★★★★
Integration	模組間 contract 不一致	100ms	中	★★★★
E2E	完整用戶流程	分鐘	高	★★★
Property-based	邊界 case 與不變性	秒	中	★★★★
Mutation	假測試（測試品質保證）	分鐘	自動	n/a (infra)
Fuzz / Snapshot	未預期 input、UI 變動	秒	中	★★★★

Type 1：Unit Tests

定義：測單一函式 / class / 純邏輯，無 IO、無 DB、無 network。

典型例子：

```
def test_calculate_discount():
    assert calculate_discount(price=100, code='SAVE10') == 90
    assert calculate_discount(price=100, code=None) == 100
    assert calculate_discount(price=100, code='INVALID') == 100
```


何時用：

- 純函式 (business logic、計算、轉換)
- Data structure 操作
- Validation 邏輯

何時不用：

- 牽涉外部依賴 (DB / API) → 用 Integration
- 跨模組互動 → 用 Integration
- UI 行為 → 用 E2E

AI 寫 unit test 的特殊優勢：

你給 AI	AI 給你
函式 signature + 1-2 個 example	完整 test suite (normal / edge / error)
函式實作	從實作 reverse engineer 出測試 (注意：要 sanity check)
Bug fix PR	對應的 regression test

Prompt 範本：

Write comprehensive unit tests for this function.
Include:
- Normal cases (typical input)
- Edge cases (boundary values, empty, very large, very small)
- Error cases (invalid input, exceptions expected)
- At least 5 distinct test cases

Use pytest. Each test should have a descriptive name.

[paste function code]

Type 2 : Integration Tests

定義：測多個模組互動、有 DB / 有 API call、但仍在單一 process 內。

典型例子：

```
def test_user_registration_flow(db, mailer):
    response = client.post('/register', json={
        'email': 'foo@bar.com',
        'password': 'secret'
    })
    assert response.status_code == 201
    # DB 真寫入
    user = db.query(User).filter_by(email='foo@bar.com').first()
    assert user is not None
    # Email 真寄出
    assert mailer.last_sent.to == 'foo@bar.com'
```

何時用：

- API endpoint 行為
- DB schema + ORM
- 多 service 互動
- Auth / permission flow

何時不用：

- 純邏輯 → Unit
- 跨 service 跨機器 → E2E
- 第三方 API 真實互動 → 用 contract test，不是 integration

重要原則：整合測試打真 DB (test container)，不要全 mock。

過去 mock 流行是因為設置真 DB 慢。現在用 Docker / testcontainers，一切都是 throwaway。一條來自 12-factor 維護人的觀察：

「Mock 通過了，但 prod migration 失敗」——這就是 mock-everywhere 的代價。

Type 3 : E2E Tests

定義：模擬真實用戶從外部操作整個系統。瀏覽器 / API client → 多 service → DB。

典型例子（用 Playwright）：

```
test('user can complete checkout', async ({ page }) => {
  await page.goto('/');
  await page.click('text=Add to cart');
  await page.click('text=Checkout');
  await page.fill('input[name=email]', 'test@example.com');
  await page.fill('input[name=card]', '4242424242424242');
  await page.click('text=Pay $99');
  await expect(page.locator('text=Thank you')).toBeVisible();
});
```

何時用：

- 關鍵用戶流程 (signup / checkout / cancellation)
- Cross-service 流程
- 看得到 / 摸得到的 UI 行為

何時不用：

- Edge case 細節 → Unit
- 模組互動 → Integration
- 慢、貴、不穩定，不該當主力

E2E 的數量原則：

5-15 個 critical journey 涵蓋核心商業流程。不要超過 **30 個**——E2E 跑太慢，flaky 率高，維護成本超過邊際效益。

Type 4 : Property-based Tests

定義：不是測「對某個 **input X**，輸出是 **Y**」，而是測「對所有 **input**，某個 **property** 永遠成立」。

典型例子（用 Hypothesis）：

```
from hypothesis import given, strategies as st
```

```
@given(st.lists(st.integers()))
def test_sort_idempotent(lst):
    # property: sort 兩次跟 sort 一次結果一樣
    assert sorted(sorted(lst)) == sorted(lst)
```

```
@given(st.lists(st.integers()))
def test_sort_preserves_length(lst):
    # property: sort 後長度不變
    assert len(sorted(lst)) == len(lst)
```

Property-based test 工具會自動生成數百個 input 試圖打破 property。

何時用：

- 數學性質明確的函式（sort、merge、compress）
- Idempotency（呼叫多次效果一樣）
- Invariant（系統不變性）
- Round-trip（encode + decode = identity）

何時不用：

- 沒有明確 mathematical property 的 UI / business logic
- 只關心特定 input → Unit

AI 寫 property-based test 的能力：★★★★。Prompt：

For this function, identify 3-5 mathematical/logical properties that should always hold. Then write Hypothesis (Python) / fast-check (JS) tests for each.

[paste function]

Type 5 : Mutation Testing

定義：測試你的測試。工具會故意改你的 code（變 + 成 -、變 < 成 <=）然後跑你的測試。如果測試沒抓到，你的測試是假的。

工具：

- Python : mutmut, mutpy
- JS : Stryker
- Java : PIT
- Go : go-mutesting

典型輸出：

```
Total mutants: 234
Killed (caught by tests): 187
Survived (NOT caught): 47    ← 假測試！
Mutation score: 80%
```

Mutation score 怎麼解讀：

分數	評估
< 50%	測試大部分是假的
50-70%	一半測試有效
70-85%	中位數
85-95%	良好
> 95%	卓越

Garry 的 90% coverage 應該對應 **mutation score $\geq 75\%$** 。否則就是 gaming。

何時用：

- Critical path 一定要跑
- Quarterly 全 codebase 跑
- PR 級別跑 diff mutation

何時不用：

- Mutation testing 很慢 ($\times 10$ test runtime)，不適合 every-commit

Type 6 : Fuzz Testing & Snapshot Testing

Fuzz

定義：用隨機 / structured-random input 砸你的函式，看會不會 crash / hang。

典型例子：

```
@hypothesis.given(st.binary())
def test_parse_doesnt_crash(data):
    try:
        my_parser.parse(data)
    except (ValueError, ParseError):
        pass # expected errors are OK
    # 但 segfault / hang / unhandled exception 就 fail
```

何時用：

- Parser、deserialization、user input handling
- 安全敏感 code

AI 寫 fuzz test：★★★★☆。Prompt:

Write fuzz tests for this parser.
Use Hypothesis to generate random bytes/strings.
Verify it never crashes (only raises expected exceptions).

[paste parser]

Snapshot

定義：把 function output 第一次跑的結果存起來（snapshot），之後每次跑跟 snapshot 比，不同就 fail。

何時用：

- UI 渲染（React component → HTML snapshot）
- 大型 JSON / YAML 輸出
- 報告生成

陷阱：snapshot 容易「**meaningless update**」——壞掉直接 `--update-snapshots` 一鍵更新，等於沒測。
人類 review 每次 snapshot 更新才有意義。

比例分配建議

90% coverage 的具體組合（典型 web service）：

70% Unit tests	← 速度快、寫得多、AI 友善
20% Integration tests	← 抓 contract 問題
5% E2E tests	← 5-15 個 critical journey
3% Property-based tests	← 關鍵函式
1% Fuzz / Snapshot	← 特殊場景

99%

外加：

- Mutation score：critical path 100%、aggregate $\geq 75\%$
 - Performance test：critical path 有 latency SLA 檢核
-

反例：常見比例錯誤

反例 1：100% E2E

「我們只寫 E2E，因為 unit test 太細」

問題： - E2E 跑超慢（30 分鐘 CI），開發者開始 skip - Flaky 高，造成 CI noise，慢慢沒人信任 - Bug 出現時找不到根因（E2E fail 通常只說「某處壞了」）

反例 2：100% Unit

「我們只寫 unit test，因為它快」

問題： - 模組間 contract 不一致時抓不到 - DB migration 不對抓不到 - API 改 schema 抓不到

反例 3：100% Mock

「所有測試都 mock 外部依賴，因為慢」

問題： - Mock 與 real 行為 drift（mock 永遠成功、real 偶爾失敗） - Refactor 時 mock 沒同步 → 假成功 - Mock pass 但 prod fail（最致命）

測試類型 vs ratchet 區域對照

回到 [Ch 04 Q2](#)：你應該在 ratchet bite hardest 的區域加最多測試。不同 **ratchet** 區用不同測試類型：

Ratchet 區	主要測試類型
業務邏輯	Unit + Property-based
Data schema	Integration + Migration test
整合層	Integration + Contract test
配置 / feature flag	E2E + Snapshot
無聲失敗	Mutation + Fuzz

對症下藥。

本章小結

類型	用途	比例
Unit	函式邏輯	70%
Integration	模組互動 / DB / API	20%
E2E	關鍵用戶流程	5%（5-15 個）
Property	數學性質	3%
Fuzz / Snapshot	特殊	1%
Mutation	測試品質保證	跑在上述之上

90% line coverage 加上 mutation score $\geq 75\%$ = 真 90%。

接下來

➡ [Chapter 06: AI 當測試作者 — 具體 SOP](#)

[← 回 README](#) · [← Ch 05](#)

第 06 章：AI 當測試作者 — 具體 SOP

核心句：「用 AI 寫測試」聽起來簡單，做不好就是「假覆蓋率」。本章給你 6 個工作流，把 AI 從「生產更多假測試的工具」變成「真把覆蓋率拉到 90% 且品質可信」的工具。

AI 寫測試的能力分布

不是所有測試類型 AI 都一樣強：

測試類型	AI 能力	為何
Unit test for pure function	★★★★★	LLM 強項：規律明確
Integration test	★★★★	需要 context (fixtures、env)
E2E test (Playwright)	★★★	需要 UI knowledge + selector 穩定性
Property-based test	★★★★	LLM 知道常見 properties
Mutation test	n/a	是 infra 不是寫測試
Fuzz test	★★★★	生成 strategies 容易
Snapshot test	★★★★★	純機械
Regression test from bug	★★★★★	LLM 很強，「重現再防」邏輯清楚

工作流 1：補測試到 90%

情境：你的 codebase 目前 65% coverage，要拉到 90%。

SOP

- Step 1: 跑 coverage 找未覆蓋的 file/line
- Step 2: 用 LLM 為每個 file 生成測試 (批次)
- Step 3: 跑測試，看 pass 率
- Step 4: 失敗的測試人類 review (通常是 LLM 誤解 implementation)
- Step 5: 跑 mutation testing 抓假測試
- Step 6: 假測試重生 (給 LLM 看為何被 mutate 殺死)

具體腳本

```
#!/bin/bash
# generate-tests.sh

# Step 1: 找未覆蓋檔案
pytest --cov=src --cov-report=json:.coverage.json
UNCOVERED_FILES=$(jq -r '.files | to_entries[] | select(.value.summary.percent_covered < 90)
| .key' .coverage.json)

# Step 2: 為每個檔案生成測試
for file in $UNCOVERED_FILES; do
    echo "Generating tests for $file..."

    # 把檔案內容 + existing tests 餵給 LLM
    EXISTING_TESTS=$(find tests/ -name "*${basename $file}.py*" -exec cat {} \;)

    claude code "
Write comprehensive pytest tests for this file to reach 90% coverage.

EXISTING TESTS (don't duplicate):
$EXISTING_TESTS

SOURCE FILE:
$(cat $file)

REQUIREMENTS:
- Use pytest
- Test all branches (not just happy path)
- Use fixtures where appropriate
- Each test has descriptive name
- Output only the test code, no explanation
" > tests/test_${basename $file}
done

# Step 3: 跑測試
pytest --cov=src --cov-fail-under=90
```

Common pitfall

❌ LLM 直接從 **implementation** 推測試——這會「通過但無意義」（測試 = implementation 的鏡像）

✅ LLM 從 **docstring / spec / behavior** 推測試——這才能抓到 implementation bug

加 prompt：

IMPORTANT: Write tests based on the docstring and intended behavior,
NOT by mirroring the implementation. If implementation has bugs,
the test should catch them.

工作流 2：每個 bug fix 都寫 regression test

情境：production 出 bug → fix → 防止再發生。

SOP

Step 1: 寫一個 failing test 重現 bug (在 fix 之前)
Step 2: 寫 fix
Step 3: 確認 test 通過
Step 4: 把 test commit 進 regression suite

AI 加速

修 bug 時 prompt :

Here's a bug report:
[paste bug description / stack trace / Sentry issue]

Here's the relevant code:
[paste code]

Tasks:
1. Write a failing pytest test that reproduces this bug (before fixing)
2. Then write the minimal fix
3. Then explain why the test would have caught this earlier

Output:
- test_<descriptive>.py (failing test)
- fix in src/<file>.py
- 1-paragraph postmortem

結果：每修一個 bug，多一個 **regression test**。Coverage 跟 quality 一起拉高。

工作流 3：PR 時自動生成 diff coverage 測試

情境：保證每個 PR 不降低 coverage。

CI 設定

```
# .github/workflows/test.yml
name: Test
on: pull_request

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # 需要 git history

      - run: pip install -r requirements.txt

      - name: Run tests with coverage
        run: pytest --cov=src --cov-report=xml

      - name: Diff coverage check
        run: |
          diff-cover coverage.xml --compare-branch=origin/main \
```

```
--fail-under=90 # diff 必須 90% 覆蓋
```

```
- name: Total coverage gate
  run: |
    coverage report --fail-under=85 # 整體不能低於 85 (現在)
```

如果 PR diff coverage 不足

提示 PR 作者用 AI 補：

```
# .github/hooks/post-commit (optional)
DIFF_COV=$(diff-cover coverage.xml --compare-branch=origin/main | grep "Coverage:" | awk
'{print $2}')
if [ $(echo "$DIFF_COV < 90" | bc) -eq 1 ]; then
  echo "⚠️ Diff coverage $DIFF_COV < 90%"
  echo "Run: claude code 'generate tests for new uncovered lines'"
fi
```

工作流 4：用 AI 抓「假測試」

情境：90% coverage 達到了，但你懷疑很多測試是「**assert True** 等級的假測試」。

三層防線

防線 1：AI code-review 測試

每個 PR 的測試檔案，跑 AI 評審：

```
claude code-review --focus=tests "
Review these test files for quality issues:
- assert True / trivial assertions
- Tests that don't actually verify behavior
- Mocked everything (mock-mock-pass)
- Tests that mirror implementation (no value)
- Missing edge cases

Files: [list]
"
```

防線 2：Mutation testing 跑批次

```
# 每週日跑全 codebase mutation
mutmut run --paths-to-mutate src/
mutmut results > mutation-report.txt

# 警報: mutation score < 75%
SCORE=$(jq '.score' mutation-results.json)
if [ $(echo "$SCORE < 75" | bc) -eq 1 ]; then
  slack-notify "#engineering" "⚠️ Mutation score dropped to $SCORE%"
fi
```

防線 3：人類 review 隨機抽樣

每週 senior engineer 隨機抽 10 個測試，跑這 prompt：

For each test below, evaluate:

1. Does it actually verify behavior? (Y/N)
2. Would it catch a real bug in the function? (Y/N)
3. If implementation had off-by-one error, would this test catch it? (Y/N)

[paste 10 random tests]

如果 < 7/10 是真測試，你的測試套基礎有問題——回去用 workflow 1 重做。

Workflow 5：從 spec / PRD 寫 E2E

情境：產品經理寫了一份 PRD，工程師要實作 + 測試。

反 patternsully：先寫 code 後補 E2E

問題：- 工程師寫完 code，testing 是「驗證 **code** 做了我寫的東西」（同義反覆）- 沒抓到「**code** 沒做 PRD 寫的東西」

Pattern：PRD → E2E → Code → Iteration

Step 1: PRD 完成

Step 2: 用 AI 從 PRD 生成 E2E test (用 Playwright / Cypress)

Step 3: E2E test 跑，全部 fail (because no code yet)

Step 4: 工程師寫 code 讓 E2E pass

Step 5: PRD 改動，E2E 同步改動

AI Prompt

Here's the PRD:

[paste PRD]

Generate Playwright E2E tests covering:

1. Happy path (the primary user journey)
2. Error scenarios (what user sees when X fails)
3. Edge cases (empty state, max state, etc.)

Tests should be:

- Independent (no shared state)
- Deterministic (no flaky timing)
- Self-cleanup (set up + tear down)

Output: Playwright TypeScript code, ready to run.

這就是 **outside-in TDD** 在 AI 時代的具體形式。

Workflow 6：AI 生成 Property-based test

情境：你有個 critical 函式，想確保所有 input 行為正確，不只是手寫的 case。

Prompt

Function:
[paste function with docstring]

Tasks:
1. Identify 3-5 mathematical/logical PROPERTIES this function should always satisfy.
2. For each property, write a Hypothesis test.
3. Explain in 1 sentence per property why it must hold.

Format:
- # Property 1: <description>
 # Why: <reasoning>
 @given(...)
 def test_property_1(...): ...

- # Property 2: ...

Example properties to consider:
- Idempotency: $f(f(x)) == f(x)$
- Commutativity: $f(a, b) == f(b, a)$
- Identity element: $f(x, \text{identity}) == x$
- Inverse: $g(f(x)) == x$
- Length preservation: $\text{len}(f(xs)) == \text{len}(xs)$
- Monotonicity: $a < b \rightarrow f(a) \leq f(b)$
- Determinism: $f(x) == f(x)$ given same input

整合：AI 測試生成的 daily SOP

把這些 workflows 組合成日常：

觸發	動作	工具
PR submit	AI 生成 diff coverage 測試	CI hook + LLM
Bug report	AI 寫 regression test	工作流 2
New PRD	AI 生成 E2E skeleton	工作流 5
Refactor critical function	AI 加 property-based test	工作流 6
每週	Mutation testing 跑批次	mutmut + cron
每月	Senior engineer 隨機抽樣 review	工作流 4 防線 3
每季	全 codebase verification audit	用 Ch 04 檢核表

AI 寫測試的 5 個鐵則

1. **AI 寫測試 ≠ 不需要 review**：每個 AI 生成的測試都要人類過 1 眼
2. **Mutation testing** 是必要的「測試品質檢核員」：沒有它，AI 寫的假測試會大量滲入
3. **Prompt** 要求「從 spec 而非 implementation」：避免測試與實作同義反覆
4. 小批次生成，不要整 **codebase** 一次餵：LLM context 越短品質越穩
5. 每個 **mutation killed by your test** = 你的測試品質指標：把這個數字當成 KPI

反例集錦

反例 1：「我用 AI 一鍵生成所有測試，coverage 90%」

問題：mutation score 可能 < 50%。一堆假測試。

解：mutation testing 必跑，作為「真假 90%」的二階檢核。

反例 2：「AI 生成的測試我都接受，因為它們 pass」

問題：通過 ≠ 有意義。可能是 assert True 或測 implementation 的鏡像。

解：每個測試 review 「這在抓什麼 bug？」答不出來就丟。

反例 3：「我用 AI mock 所有外部依賴」

問題：mock 永遠成功，prod 偶爾失敗。

解：用 testcontainers 跑真 DB / 真 service，AI 只 mock 慢的、不穩的、不重要的。

本章小結

工作流	觸發	結果
1. 補測試到 90%	一次性 migration	65% → 90% in 3 週
2. Bug fix → regression	每次 bug	防止 regression
3. PR diff coverage	每個 PR	不降低 coverage
4. 抓假測試	每週 / 隨機	保品質
5. PRD → E2E	新 feature	Outside-in 開發
6. Property-based	Refactor	數學保證

接下來

 [Chapter 07: 反模式 — 假測試陷阱](#)

[← 回 README](#) · [← Ch 06](#)

第 07 章：反模式 — 假測試陷阱

核心句：80% 覆蓋率比 60% 危險——因為它讓你以為有防線。本章列 10 個 anti-patterns，幫你識別「看起來像測試但實際上是裝飾」的陷阱。

反模式 #1 : assert True

❌ 錯：

```
def test_calculate_price():
    result = calculate_price(items=[item1, item2])
    assert True # 啊?沒測什麼
```

為什麼壞：執行了 code，但沒驗證任何行為。Coverage 漂亮，零價值。

識別信號：- assert True - assert 1 == 1 - assert response is not None (只測沒 crash)

✅ 正解：

```
def test_calculate_price():
    result = calculate_price(items=[
        Item(price=100), Item(price=200)
    ])
    assert result == 300
```

反模式 #2 : 測試是 Implementation 的鏡像

❌ 錯：

```
# implementation
def calculate_tax(price):
    return price * 0.08

# test
def test_calculate_tax():
    assert calculate_tax(100) == 100 * 0.08 # ← 同一個 0.08
```

為什麼壞：如果 0.08 應該是 0.10 (程式 bug)，測試也是 0.08，永遠抓不到。測試跟著程式一起錯。

✅ 正解：用 explicit expected value：

```
def test_calculate_tax():
    # 期望: 8% 稅率 (從 spec)
    assert calculate_tax(100) == 8.0
    assert calculate_tax(50) == 4.0
    assert calculate_tax(1000) == 80.0
```

如果 spec 改 tax rate，你只改 test 對應的 expected，不會自動跟程式錯。

反模式 #3 : Mock 整個世界

❌ 錯：

```
def test_user_signup(mock):
    mocker.patch('db.User.save')          # mock DB
    mocker.patch('mailer.send')           # mock email
    mocker.patch('redis.set')             # mock cache
    mocker.patch('sentry.capture')         # mock error tracker
    mocker.patch('analytics.track')        # mock analytics

    user_signup(email='foo@bar.com', password='x')

    db.User.save.assert_called_once()      # 只驗證「**call 過了**」
```

為什麼壞：你測的不是「**user signup 正確**」，是「這些 **mock 被呼叫了**」。實際 DB schema 改了、real mailer 改了，這測試還是 pass。Mock pass，prod fail。

✅ 正解：用 testcontainers 跑真 DB + 真 service：

```
def test_user_signup(db_container): # 真 PostgreSQL container
    user_signup(email='foo@bar.com', password='x')

    # 從真 DB 查
    user = db_container.session.query(User).filter_by(
        email='foo@bar.com'
    ).first()
    assert user is not None
    assert user.password_hash != 'x' # 確實被 hash 了
```

Mock 是工具不是教義。Mock 慢的、外部的、隨機的；不要 mock 你的 schema、你的核心邏輯。

反模式 #4：超大 Test，沒有 isolation

❌ 錯：

```
def test_full_workflow():
    # 30 行 setup
    user = create_user(...)
    org = create_org(user)
    project = create_project(org)
    issue = create_issue(project)

    # 50 行 action
    add_comment(issue, ...)
    assign_user(issue, ...)
    update_status(issue, ...)

    # 20 行 assert
    assert issue.comments[0].text == ...
    assert issue.assignee == ...
    assert issue.status == ...
```

為什麼壞：- 任一行壞，整個 test fail - 不知道是哪個 action 出問題 - 失敗時 debug 1 小時起跳 - LLM 也理解不了「這個 test 到底在測什麼」

✅ 正解：每個 test 一個 action：

```
def test_add_comment(issue_factory):
    issue = issue_factory()
    add_comment(issue, 'hello')
    assert issue.comments[0].text == 'hello'

def test_assign_user(issue_factory, user_factory):
    issue = issue_factory()
    user = user_factory()
    assign_user(issue, user)
    assert issue.assignee == user
```

... 各自獨立

Test 一行 setup、一行 action、一行 assert 是聖經。

反模式 #5：Skipping Tests 卻不刪

❌ 錯：

```
@pytest.mark.skip(reason="flaky, fix later")
def test_complex_scenario():
    ...

@pytest.mark.xfail
def test_another_thing():
    ...
```

為什麼壞：- Skip 的測試從不被刪除也不被修，永遠在 codebase 裡裝樣子 - Coverage 報告仍可能算入（看工具） - 新工程師看到 skip 不知道是不是還有意義 - 跟 assert True 一樣等於沒測

✅ 正解：- 一週內修不好 → 刪掉 - 真的需要保留 → 加 **expiry** 與 **owner**：

```
@pytest.mark.skip(
    reason="DB locked race condition - tracked in JIRA-123",
    # SKIP_EXPIRES: 2026-06-01
    # OWNER: jane@example.com
)
def test_complex_scenario():
    ...
```

跟 CI 整合：超過 expiry 還 skip → CI fail，強迫處理。

反模式 #6：用 try/except 包整個 Test Body

❌ 錯：

```
def test_thing():
    try:
        result = my_function()
        assert result == expected
    except Exception:
        pass # 包住所有錯誤
```


為什麼壞：你的 test 永遠 pass，不管 my_function 是不是真壞了。

✅ 正解：只 catch 預期的 exception，並斷言它確實發生：

```
def test_thing_raises_on_invalid():
    with pytest.raises(ValueError, match="invalid input"):
        my_function(invalid_arg)
```

反模式 #7：依賴 Test 執行順序

❌ 錯：

```
def test_1_create_user():
    global user_id
    user_id = create_user('foo')
    assert user_id

def test_2_login_user():
    response = login(user_id) # ← 依賴 test_1 跑過
    assert response.status == 200
```

為什麼壞：- 換順序就壞 - 平行跑就壞 - 沒辦法 isolate debug 單一 test

✅ 正解：每個 test 自己 setup（用 fixture 共享）：

```
@pytest.fixture
def user():
    return create_user('foo')

def test_login(user):
    response = login(user.id)
    assert response.status == 200
```

反模式 #8：Flaky test 容忍症

❌ 錯：CI 有 5% test flaky。團隊「習以為常」，看到 fail 就 retry，過了就 merge。

為什麼壞：- 真 bug 跟 flaky 混在一起，分不清 - 信任度崩盤——CI fail 大家先 retry - 慢慢更多 test 變 flaky，但沒人警告 - 直到某天 prod 壞了，回查發現原來那個 flaky test 真的在抓 bug

✅ 正解：Zero tolerance

```
# 每個 PR 跑 3 次，3 次都 pass 才算 pass
pytest --reruns=0 --maxfail=1 # 嚴格模式

# 偵測 flaky
pytest --count=10 # 跑 10 次，有任何不一致就 fail
```

Flaky test 出現：1. 立即 isolate（mark + 報警，不直接 disable）2. 24 小時內 root cause 3. 修不好就刪

反模式 #9：以 LoC 為 Test KPI

✗ 錯：「這個 sprint 我們寫了 500 行 test」當成功標記。

為什麼壞：- 鼓勵寫多而非好 - AI 一秒鐘可以寫 5000 行 garbage test - LoC 跟「抓到的 bug 數」沒關係

✓ 正解：用這些 KPI：- **Mutation score**（測試品質）- **Regression test count**（每修一個 bug 加一個）- **Critical path coverage**（高風險區 100%）- **CI confidence**（綠燈才能上 prod 的信任度）

LoC 不要當 KPI。

反模式 #10：「我們有 manual QA 所以不需要這麼多自動測試」

✗ 錯：「Manual QA team 跑 regression suite，自動測試 60% 就夠」

為什麼壞：- Manual QA 一週只能跑 1-2 個完整 regression cycle - AI agent 一週可以塞 50 個 PR - 數學上 manual QA 永遠追不上

✓ 正解：把 manual QA 角色升級：

Manual QA 從做	升級成做
跑 regression（重複勞動）	設計新測試（高槓桿）
手動驗證 happy path	寫 exploratory testing tool
寫 bug report	寫 root cause analysis + 防 regression test
Test 執行	Test architecture

Manual QA 沒有消失，是被「升維」——做機器做不了的事。

反模式對照速查

#	反模式	識別信號	解法
1	assert True	沒有 assertion	用 explicit value
2	Implementation 鏡像	Test 跟 code 用同 constant	用 spec-derived value
3	Mock 整個世界	一堆 patch	用 testcontainers
4	巨型 test	100+ 行	拆成單一 action
5	不刪的 skip	@pytest.mark.skip 沒 expiry	加 expiry + owner
6	try/except 包 test	bare except	用 pytest.raises
7	依賴順序	global state	用 fixture
8	Flaky 容忍	retry 文化	Zero tolerance

#	反模式	識別信號	解法
9	LoC 當 KPI	周報講行數	用 mutation score
10	Manual QA 替代	60% 覆蓋率自滿	升級 QA 角色

自我檢查腳本

把這個跑你 codebase 一遍：

```
#!/bin/bash
# detect-anti-patterns.sh

echo "=== 反模式 #1: assert True / 平凡 assertion ==="
grep -rn 'assert True|assert 1 == 1|assert None is None' tests/ || echo "✓ none"

echo "=== 反模式 #5: skip 沒 expiry ==="
grep -rn 'pytest.mark.skip|@skip' tests/ | grep -v 'SKIP_EXPIRES' || echo "✓ all good"

echo "=== 反模式 #6: bare except ==="
grep -rn 'except:|except Exception:\s*pass' tests/ || echo "✓ none"

echo "=== 反模式 #7: global state in tests ==="
grep -rn '^global ' tests/ || echo "✓ none"

echo "=== Test 行數分布 (找太大的 test) ==="
awk '/^def test_{n=NR; name=$0} /^def [^t]/{if(n)print NR-n, name; n=0}' tests/*.py | sort
-n -r | head -10

echo "=== 跑 mutation testing (基礎品質指標) ==="
mutmut run 2>&1 | tail -3
```

本章小結

反模式	危害程度	修復難度
#1 assert True	高	低
#2 Impl 鏡像	致命 (看不到 bug)	中
#3 Mock 全世界	致命 (mock pass prod fail)	中
#4 巨型 test	中 (debug 痛苦)	低
#5 不刪 skip	中 (累積技術債)	低
#6 try/except 包 test	高	低
#7 順序依賴	中	低
#8 Flaky 容忍	致命 (信任度崩盤)	高
#9 LoC KPI	中	低
#10 Manual QA 替代	中	中

修這 10 個之前，講 90% 覆蓋率都是空話。

接下來

[➡ Chapter 08: 12 週實施路線圖](#)

[← 回 README](#) · [← Ch 07](#)

第 08 章：12 週實施路線圖 — 從 65% 拉到 90%

核心句：把 Garry Tan 的 90% 目標拆成 12 週可執行計畫。每週有 deliverable、有 owner、有檢核點。把這份當作下個 quarter 的工程 OKR。

前提假設

條件	數值
目前 line coverage	65%
Mutation score	未測量 / < 50%
工程團隊	10-30 人
主要技術 stack	Python/TypeScript/Go (任一)
AI 工具預算	\$1000-3000/月
目標	第 12 週末 ≥ 90% line, ≥ 75% mutation

路線圖總覽

- Week 1-2: 測量 + 立 baseline
- Week 3-4: 攻擊 Critical Path (最高優先)
- Week 5-6: Business Logic 補測試
- Week 7-8: Integration & E2E
- Week 9-10: Mutation testing + 假測試清除
- Week 11: CI hardening + diff coverage gate
- Week 12: 全面 audit + 90% 確認

Phase 1：測量 + 立 baseline (Week 1-2)

目標：知道現狀，定 KPI。

Week 1: 測量

- ☐ 跑 coverage report，記錄每個 **module** 的覆蓋率
- ☐ 跑 mutation testing 在 critical path (至少 3 個重要 module)
- ☐ 列出 [Ch 04](#) 9 項 verification 維度的當前分數

- ☐ 列出過去 12 個月 P0/P1 incident 對應的系統
- ☐ 問所有 senior：「不敢動」清單

Week 2: 立 baseline

- ☐ 寫一份 1 頁 report 給 leadership：「Coverage 現狀 + 12 週計畫」
- ☐ CI 加 `--fail-under=65`（從現狀）並 commit
- ☐ 指派 Quality Lead（有阻擋 PR 權力）
- ☐ 訂 90% 目標、訂 mutation score $\geq 75\%$ 目標
- ☐ 公開 dashboard（顯示 coverage、mutation、reg test count）

Phase 1 deliverable

1. Coverage baseline report
 2. CI 強制執行 `--fail-under=65`
 3. Quality Lead appointed
 4. Public dashboard
-

Phase 2：攻擊 Critical Path（Week 3-4）

目標：高風險系統 100% 覆蓋。

從 [Ch 04 Q2](#) 找出 ratchet bite hardest 的 3 個系統，這兩週只攻它們。

Week 3-4: Critical Path 100%

每個 critical 系統：- [] 列出所有 entry function（API endpoint、business logic 入口）- [] 用 AI 寫 unit tests（[Ch 06 工作流 1](#)）- [] 人類 review 每個 test - [] 跑 mutation testing，補假測試（[工作流 4](#)）- [] 確認該系統覆蓋率 $\geq 95\%$, mutation $\geq 80\%$

Phase 2 deliverable

- 3 個 critical 系統覆蓋率 100%
 - Mutation score $\geq 80\%$
 - 整體覆蓋率從 65% $\rightarrow$ 70%
-

Phase 3：Business Logic 補測試（Week 5-6）

目標：非 critical 但常改的 business logic 拉到 90%。

Week 5: Identify

- ☐ 列出 `git log --since="1 year ago" --name-only` 改最多次的 file
- ☐ 取 top 50%（這些是常改的、需要 regression 保護）

- ☐ 排除已在 critical path 的

Week 6: AI Mass Generation

- ☐ 用 [workflow 1](#) 批次生成
- ☐ 每天 50-100 個 test
- ☐ 人類 review SLA 24 小時
- ☐ Reject 比例不能 > 30% (太高代表 prompt 還沒調好)

Phase 3 deliverable

- 高頻 file 覆蓋率 $\geq 90\%$
 - 整體 65% $\rightarrow$ 78%
-

Phase 4 : Integration & E2E (Week 7-8)

目標：模組間 contract + 關鍵用戶流程。

Week 7: Integration

- ☐ 列出所有 API endpoint
- ☐ 每個 endpoint 寫 integration test (真 DB, 不是 mock)
- ☐ 用 testcontainers / docker-compose

Week 8: E2E

- ☐ 列出 5-15 個 critical journey
- ☐ 用 Playwright / Cypress 寫 E2E
- ☐ 用 [workflow 5](#) 從 PRD 生成
- ☐ CI 跑 E2E (限制: parallel + retries=0)

Phase 4 deliverable

- API endpoint 100% integration covered
 - 5-15 個 critical E2E journey 跑得穩
 - 整體 78% $\rightarrow$ 85%
-

Phase 5 : Mutation + 假測試清除 (Week 9-10)

目標：mutation score $\geq 75\%$ ，徹底清除假測試。

Week 9: Mutation 全面跑

- ☐ 跑 mutation testing 在整個 codebase

- ☐ 列出 surviving mutants（沒被測試殺死的）
- ☐ 把 mutation result 餵 LLM，請它改善 existing tests

Week 10: 假測試手動 audit

- ☐ Senior engineer 隨機抽 100 個 test
- ☐ 跑 [Ch 07](#) 的反模式檢查腳本
- ☐ 刪掉 / 修復假測試（預期 10-20% 需要動）

Phase 5 deliverable

- Mutation score: 50% → 75%
 - Anti-pattern detection 全 pass
 - 整體 85% → 88%
-

Phase 6 : CI 加固 + Diff Coverage Gate (Week 11)

目標：守住 90%，不退步。

Week 11

- ☐ CI 加 diff coverage gate：每 PR 新行 ≥ 90% 覆蓋
- ☐ CI 加 mutation gate：critical path 每 PR mutation ≥ 80%
- ☐ CI 加 anti-pattern 檢查（[Ch 07 腳本](#)）
- ☐ 升 --fail-under 從 65% → 88%
- ☐ 公告：超過 88% 的 module 才能 merge，例外要 escalate

Phase 6 deliverable

- CI 鎖住「不能退步」
 - 全組織意識到「測試是 first-class」
-

Phase 7 : 最終 audit + 90% confirmation (Week 12)

目標：證明 90% 達成且有意義。

Week 12

- ☐ 整體 coverage 跑一次最終 report
- ☐ Mutation score 跑一次最終 report
- ☐ [Ch 04](#) 9 維度重新打分（目標：≥ 80）
- ☐ 跟 Quality Lead 對 critical path：100% line, ≥ 80% mutation
- ☐ CI --fail-under=90
- ☐ 寫 12 週 retrospective：什麼有效、什麼沒效、Q3 計畫

☐ 慶祝 + 給團隊 credit

Phase 7 deliverable

- 整體 $\geq 90\%$ line, $\geq 75\%$ mutation
 - Critical path 100% line, $\geq 80\%$ mutation
 - CI 強制 90%
-

週度 standup template

每週 Quality Lead 帶這個 standup：

Quality Weekly – Week N

****Coverage****

- 整體：__% (本週 +/- %)
- Critical path: __%
- Mutation: __%

****This week****

- Wins:
- Blockers:
- AI gen ratio (人類 review pass rate): __%

****Risks****

- Module X 覆蓋率仍 $< 60\%$, owner: ____
- Y 個 flaky test 待處理

****Next week****

- 攻擊 module Z
- 完成 ____

****Help needed****

- ____
-

如果你預算 / 人力更少

縮短版（6 週）：

Week	行動
1	測量 + 列 critical 系統
2-3	AI 補 critical 系統測試
4	Mutation testing + 假測試清除
5	Integration + 1-2 個 critical E2E
6	CI gate + audit

目標調整：從 90% 改 80%（但 critical path 仍 100%）。

如果你預算 / 人力更多

加速版（8 週達成 + 後續強化）：

Week	額外項目
1-2	+ 全 codebase mutation testing 跑出 baseline
3-4	+ 平行 5 個 critical 系統
5-6	+ Performance test + chaos engineering
7-8	+ 全 codebase diff mutation in CI

達成 90% 之後做什麼

繼續維持是一回事，更高層次的工作：

- 1. **Verification observability**：在 production 直接 inject 故障測試（chaos engineering）
- 2. **Contract testing** 跨 service
- 3. **Performance regression** budget
- 4. **Security test** 整合（SAST / DAST 進 CI）
- 5. **AI** 測試品質的 **ML** 評估（更 advanced 的 mutation 變體）

但這些都建立在「已有 90% 真品質覆蓋率」之上。先做好 baseline，再玩進階。

預算估算

項目	12 週成本
Quality Lead（兼任 30% time）	內部資源
AI 工具（Claude / Cursor）	\$200-500/月 × 12 週 = \$600-1500
Mutation testing infra（CI 額外計算）	\$200-500/月
Testcontainers infra	通常已有
人類 review 時數（4 senior × 4 hrs/week × 12 週）	\$30K
總計	~\$35-45K

ROI 比較：一個 P0 incident 平均 \$50K-200K（service downtime + 工程師時間 + customer churn）。**12 週** 投資 < 一次大 **incident** 損失。

本章小結

Phase	週次	目標
1. 測量 + baseline	1-2	知現狀

Phase	週次	目標
2. Critical path	3-4	高風險系統 100%
3. Business logic	5-6	常改 file 90%
4. Integration + E2E	7-8	API + journey
5. Mutation + 清假	9-10	品質保證
6. CI 加固	11	守 90%
7. 最終 audit	12	確認達成

12 週後：**90% line, 75% mutation, CI 強制**。組織完成 AI 時代的驗證能力升級。

接下來

[➡ Chapter 09: 跟 12-Factor Agents 的關聯](#)

[← 回 README](#) · [← Ch 08](#)

第 09 章：跟 12-Factor Agents 的關聯

核心句：Garry Tan 的「90% test coverage」跟 Dex Horthy 的「12-factor agents」乍看談不同問題。實際上它們是「**LLM 時代 production-grade 軟體**」這枚硬幣的兩面——12-factor 是建造側、90% 是驗證側。沒有任一面，AI 應用都上不了 customer-facing production。

兩條觀念的關係

觀念	提出者	解決問題
12-factor agents	Dex Horthy	怎麼建造可控的 LLM 應用
90% test coverage	Garry Tan	怎麼驗證 AI 產出的軟體

12-factor 是「寫對」，**90%** 是「確認沒寫錯」。一個是 input quality、一個是 output quality。

為什麼這兩條必須同時存在

只有 12-factor 而沒有 90%：- Agent 架構漂亮、可控、可暫停 - 但你不知道它在 **production** 會不會壞——AI agent 寫的 deterministic 程式碼可能有 bug 你沒抓到 - 結果：可控的 agent 還是會出 incident

只有 90% 而沒有 12-factor：- Agent 寫的 code 都被測試覆蓋 - 但 **agent** 本身是黑盒，走火入魔時沒人能介入 - 結果：production agent 跑 5 分鐘就 spin out

兩條一起做，你才得到「**production-grade AI 應用**」。

對應映射

Garry Q1 (verification coverage) ↔ 12-factor Factor 9 (Compact Errors)

Garry Q1: What is our verification coverage on AI outputs?

Factor 9: 把錯誤壓進 context window，讓 LLM 自癒

兩條都在問「錯誤怎麼被抓到 + 修復」。

- **Factor 9** 處理「運行時錯誤」：tool call 失敗時，LLM 看到 stack trace 後嘗試自我修復
- **Garry Q1** 處理「部署前錯誤」：測試體系抓到 bug，在 PR 階段就 reject

兩個都是「錯誤捕獲」，只是在不同 stage。

Garry Q2 (ratchet bite hardest) ↔ 12-factor Factor 10 (Small Focused Agents)

Garry Q2: Where does the ratchet bite hardest?

Factor 10: Small, focused agents (3-10 step)

兩條都在處理「複雜度爆炸」。

- **Factor 10** 從結構上避免複雜度：把 agent 拆小，每個只處理一件事
- **Garry Q2** 從驗證上對抗複雜度：找出複雜度最大的區域，集中防線

結構限制 + 驗證強化，是對抗複雜度的兩支腳。

Garry Q3 (who owns) ↔ 12-factor Factor 7 (Contact Humans with Tools)

Garry Q3: Who owns catching regressions?

Factor 7: 用 tool call 聯絡人類（人類介入機制）

兩條都在處理「人類在 loop 的角色」。

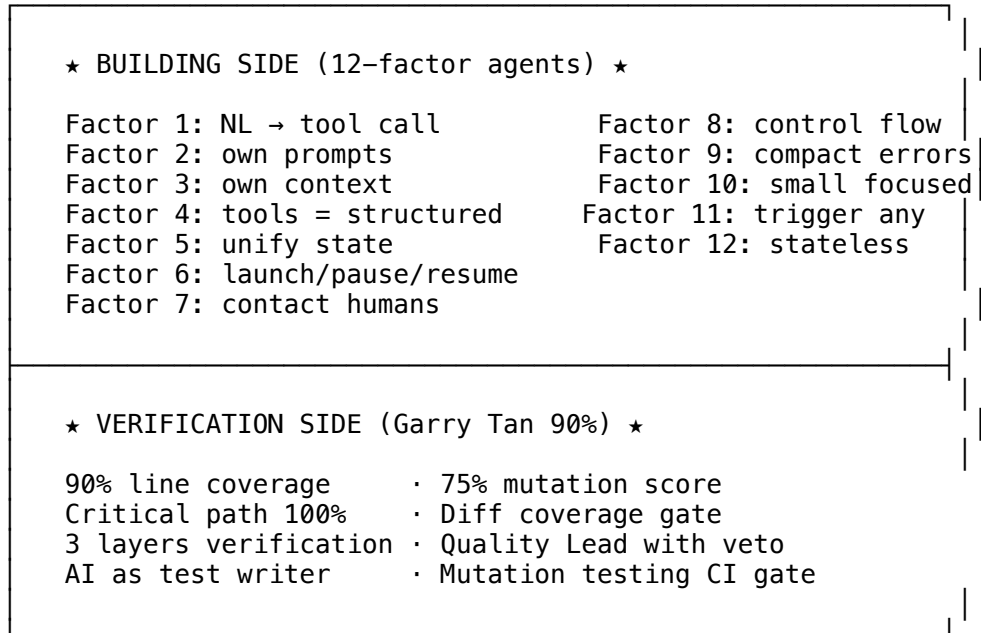
- **Factor 7** 設計運行時人類介入：agent 跑到關鍵點主動找人類
- **Garry Q3** 設計組織人類介入：明確誰負責抓 regression

人類不是裝飾，是 **critical infrastructure**。

完整 production architecture

把兩條觀念合成完整架構：

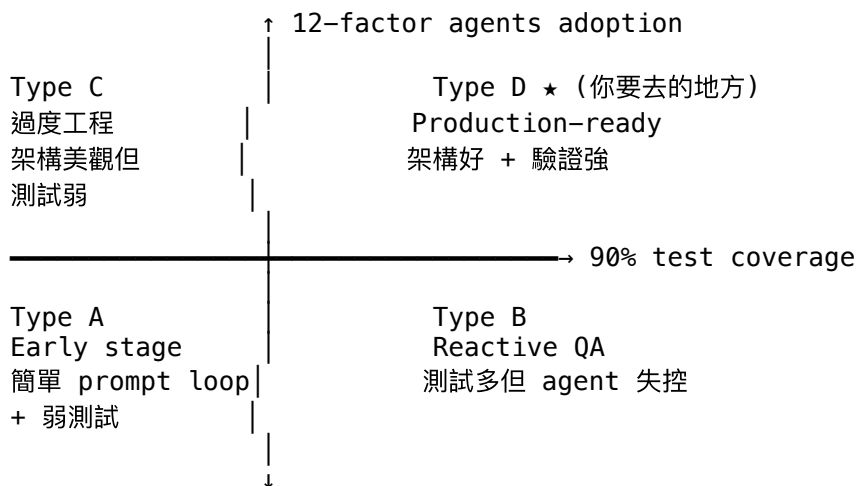
Production-Grade AI Application



↓
PRODUCTION-GRADE AI APPLICATION

工程組織的 4 種狀態

根據兩條原則的完成度，組織分 4 種：



多數 AI 新創 **stuck** 在 **Type A** (沒測試也沒架構) 部分大公司在 **Type C** (架構漂亮但沒測試) 少數成熟公司在 **Type B** (測試多但 **agent** 鬆散) 極少數在 **Type D** (兩者都做)

Garry + Dex 的方法論合起來，就是把你從任何起點推到 **Type D** 的 **playbook**。

整合範例：12-factor agent 的 verification 體系

把兩條觀念套到一個具體 **deploybot** 案例：

12-factor 設計

標準 12-factor agent 架構

```
class DeployAgent:
    def handle_next_step(self, thread: Thread):
        while True:
            next_step = self.llm.determine_next_step(thread_to_prompt(thread))

            if next_step.intent == 'deploy_backend_to_prod':
                # high-stakes: 需要人類審批
                self.request_human_approval(next_step)
                self.save_thread(thread)
                break
            elif next_step.intent == 'list_git_tags':
                tags = fetch_git_tags()
                thread.events.append({'type': 'list_git_tags_result', 'data': tags})
                continue
            # ... 其他 case
```

Garry 90% 驗證體系

對應的測試 suite :

Unit tests

```
def test_handle_next_step_deploy_intent():
    """高 stakes deploy intent 必須 break loop 等人類"""
    agent = DeployAgent(mock_llm_returning('deploy_backend_to_prod'))
    thread = Thread()
    agent.handle_next_step(thread)
    assert mock_request_human_approval.called
    assert thread.is_saved
```

```
def test_handle_next_step_list_tags_intent():
    """List tags 同步執行不 break"""
    agent = DeployAgent(mock_llm_returning('list_git_tags'))
    thread = Thread()
    # 應該 continue loop 不 break
    ...
```

Integration tests

```
def test_full_deploy_flow_with_real_db(db_container):
    """完整 deploy flow: trigger → LLM decisions → human approval → execute"""
    ...
```

E2E tests

```
def test_slack_triggered_deploy_e2e(playwright):
    """從 Slack message 觸發到 deploy 完成的完整 journey"""
    ...
```

Property-based tests

@given(thread_strategy)

```
def test_thread_serialization_round_trip(thread):
    """Property: thread serialize + deserialize = identity (Factor 5/12)"""
    serialized = json.dumps(thread.to_dict())
    restored = Thread.from_dict(json.loads(serialized))
    assert restored == thread
```

```
# Mutation testing
# 跑 mutmut，確認 critical control flow 的 mutation 都被殺死
```

這套測試對應 **12-factor** 的每個 **factor**： - Factor 8 (control flow) → unit tests for switch cases - Factor 5 (state) → property-based serialization round-trip - Factor 7 (human) → integration test 確認 approval 路徑 - Factor 6 (pause/resume) → integration test 確認 save/load thread - Factor 11 (trigger anywhere) → E2E test 從 Slack 觸發

1+1 > 2 的綜效

場景	只有 12-factor	只有 90%	兩者皆有
Agent 走火入魔	Factor 10 防範 + Factor 9 自癒	測試抓不到 runtime 問題	★ 結構限制 + 運行時自癒 + 測試保底
Bug 進 production	架構好但仍可能漏	測試抓到	★ 雙重防線
Human-in-loop 失效	Factor 7 結構保證	沒這概念	★ 結構 + integration test 驗證
Refactor 風險	Factor 12 純函數性幫助	測試保底	★ 結構 + 測試
Cross-team handoff	架構文件化	Test 是文件	★ 雙重文件

給工程 leadership 的綜合建議

- 不要選邊站：兩條一起做。如果預算只夠做一條，先做 **90%**（短期 **ROI** 更高），同時規劃 **12-factor** 的 **phase-in**。
- Headcount** 配置：
 - 50% 開發團隊（寫 feature with AI）
 - 30% 平台 / quality team（負責 12-factor architecture + 90% gating）
 - 20% reliability / SRE（production verification）
- 季度 **OKR** 配對：
 - Q1: 12-factor adoption（agent code 重構）+ Coverage 65% → 75%
 - Q2: Factor 5/6/7 完成 + Coverage 75% → 85%
 - Q3: Factor 9/10 完成 + Coverage 85% → 90% + Mutation 75%
 - Q4: Audit + 升級到 Type D
- 預算 **ratio**：
 - AI tool 預算（包含寫測試 + 寫 feature）：\$2-5K/工程師/年
 - Verification infra：\$1-3K/工程師/年
 - 加起來 < 5% 工程薪資總和

終極論點

Garry Tan 跟 Dex Horthy 是同一場戰役的兩個將軍：

- Dex 守住「LLM 應用怎麼建」
- Garry 守住「LLM 應用怎麼驗」

兩條都贏，你的組織就贏。任何只贏一條的組織，最終會被同時做到兩條的對手取代。

“The companies that win the next 24 months won’t be the ones with the fastest agents...They’ll be the ones with the strongest guardrails.” — Garry Tan

把 12-factor 當「建造 guardrail 的圖紙」，90% 當「guardrail 的承重測試」。兩個都做完，你才真正擁有 **guardrail**。

結尾

恭喜你讀完整份 Garry Tan 複雜度棘輪 + 90% 測試覆蓋率精解。

一句話帶走

AI agent 寫程式比人類快 10 倍。如果驗證能力沒同步 $\times 10$ ，你就被複雜度棘輪追上了。**90%** 測試覆蓋率（含 **mutation $\geq 75\%$** ）是 AI 時代的新基線。

該做的第一步

1. 跑 coverage report，看你現在數字
 2. 看 [Ch08 路線圖](#) 的 Week 1
 3. 設一個 12 週後的 90% 目標
 4. 開始
-

[← 回 README](#)